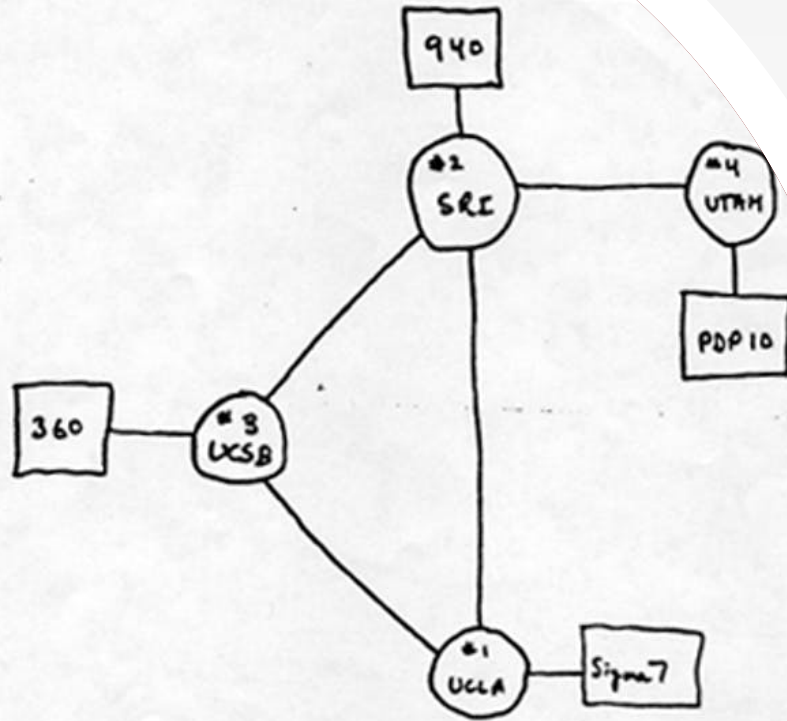


INF1005

Topics for Week 01:

- The Internet
- The World Wide Web
- HTML5





THE ARPA NETWORK

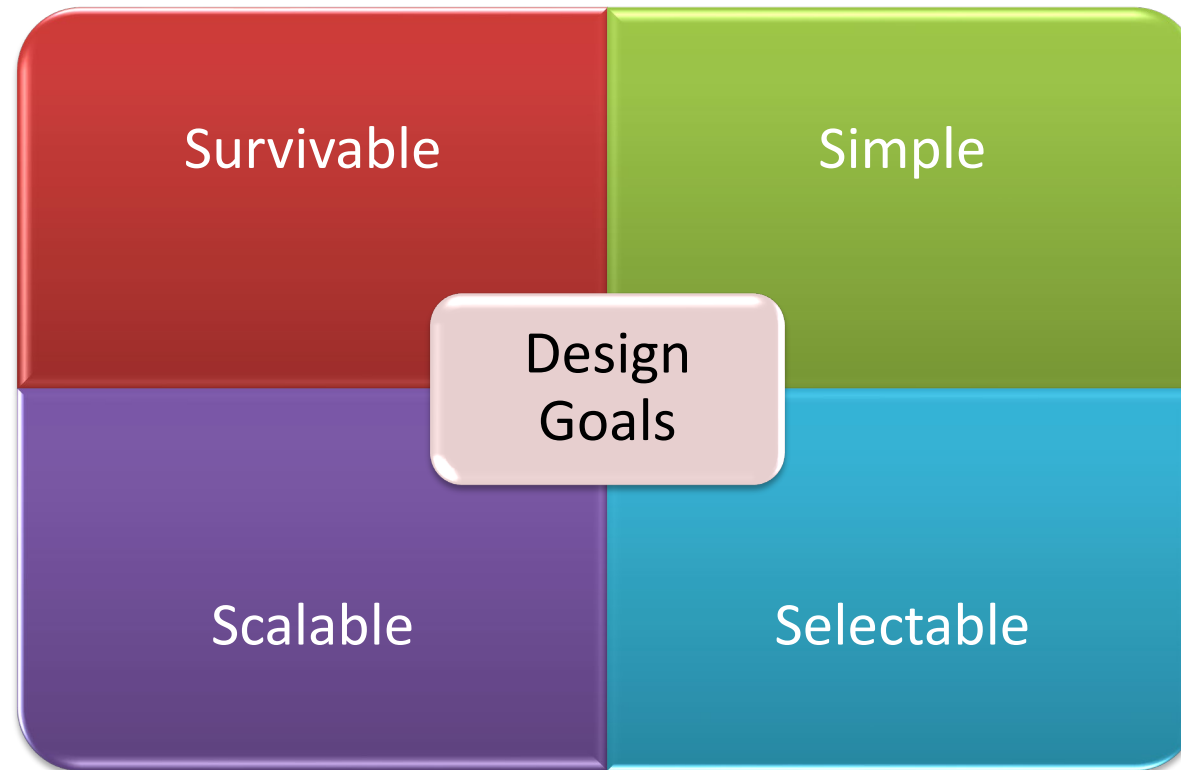
DEC 1969

4 NODES

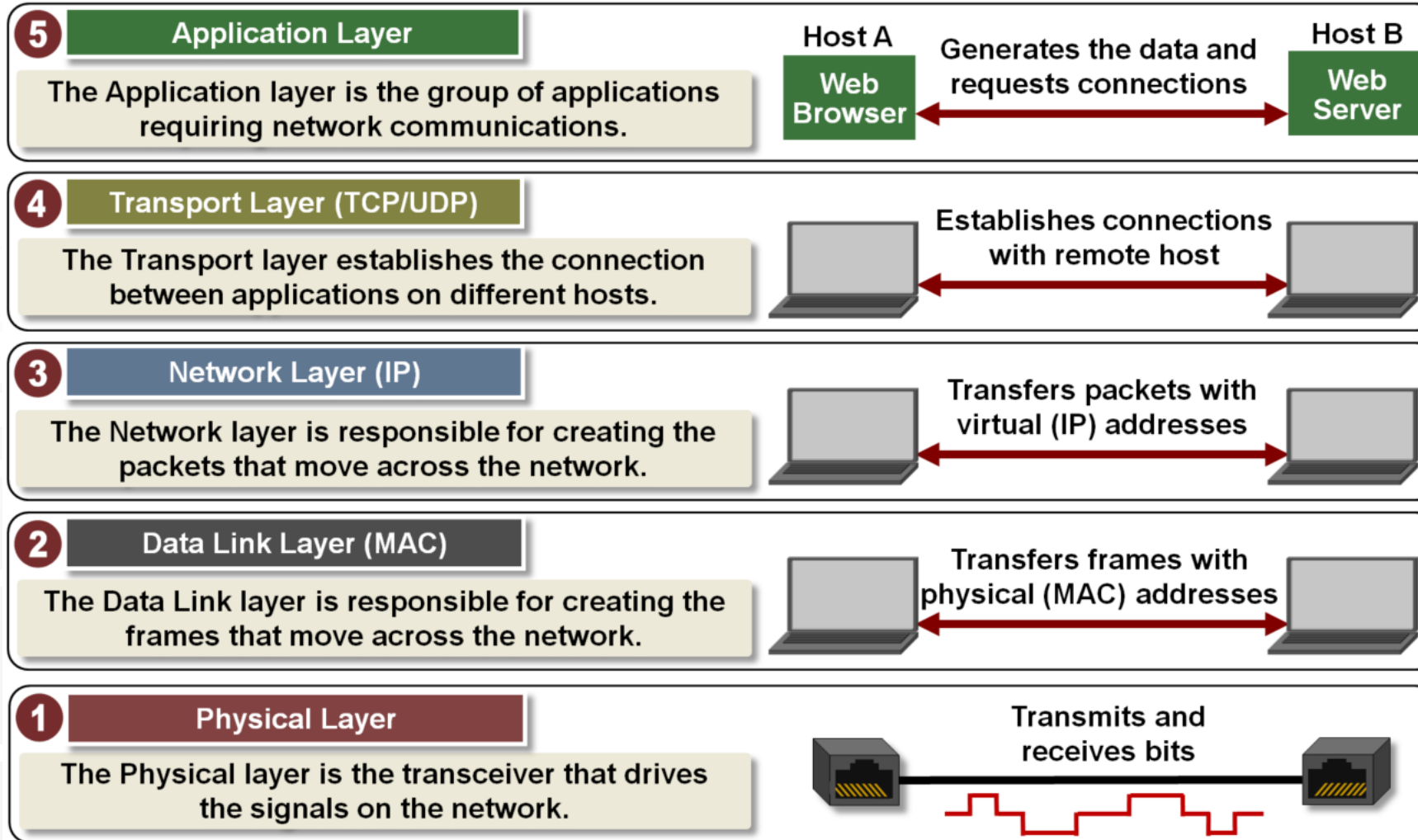
THE INTERNET

- History
- Architecture
 - TCP/IP
- Client-Server vs. Peer-to-Peer
- DNS
- Design Goals
- Benefits

ARPANET Design Goals



TCP/IP 5-LAYER MODEL



Domain Name Service (DNS)

- Translation of hostnames to and from IP addresses
- Why hostname e.g. `www.amazon.com`
 - Easy to remember
 - Re-locatable
 - Same web site available in multiple hosts
- DNS resolver performs two functions:
 - *gethostbyname()*, converts name to IP address
 - *gethostbyaddr()*, converts IP address to name
- A database of mapping is cached

Hostname	IP Address
singaporetech.edu.sg	158.122.11.23 (IPv4)
apple.com	2620:149:af0::10 (IPv6)

Benefits of the Internet

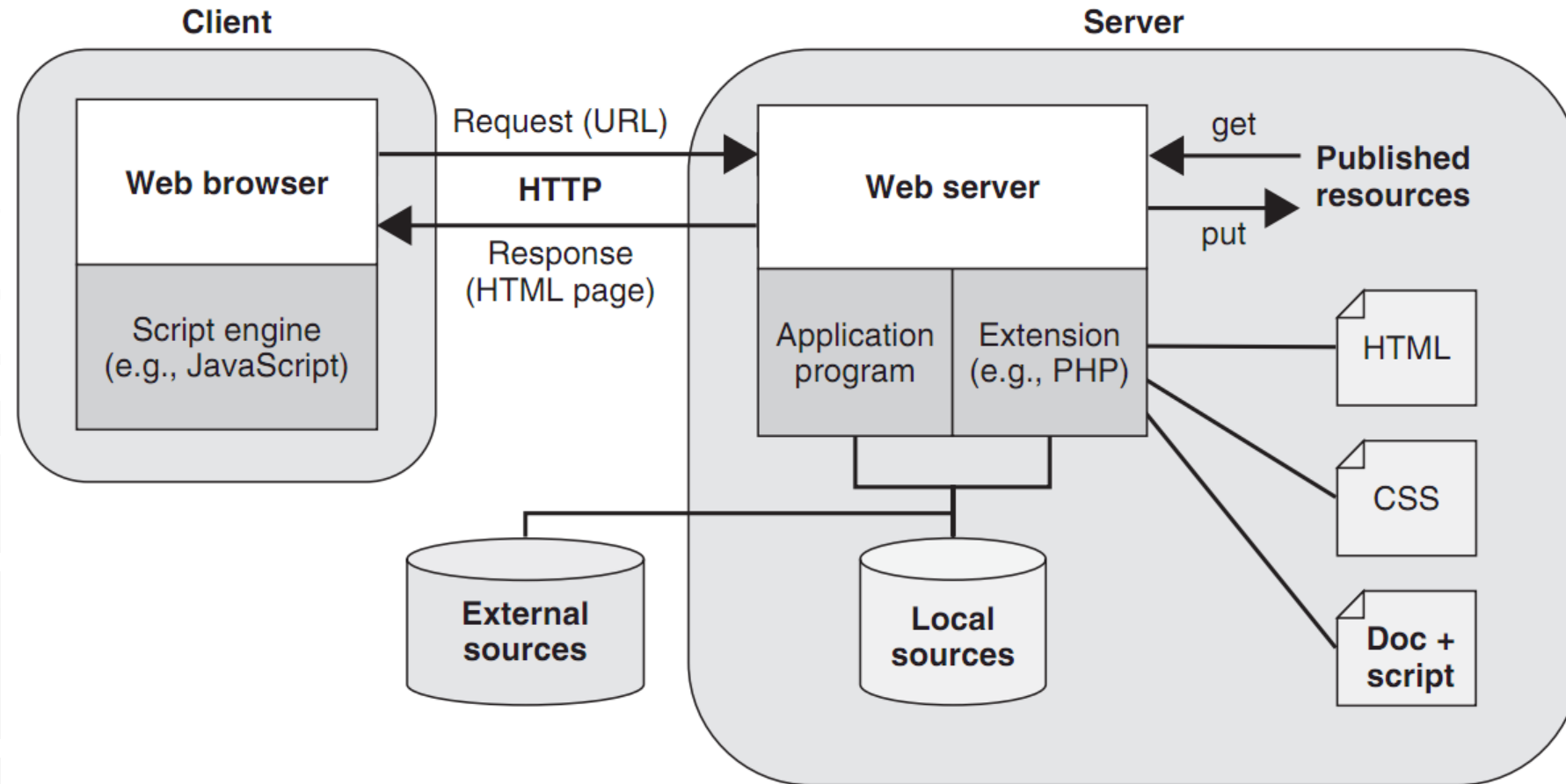
- Fully Decentralized makes it highly robust
 - A part of non-functioning network does not affect others
- Intelligence is at edge, while the routing is simple
 - Makes it scalable to heavy traffic
- Low cost
 - Very affordable to get connected to the internet
 - Virtual Private Network allows companies to use Internet to create “private” networks
- Many established services e.g. E-commerce, social networks, Wikipedia, etc.

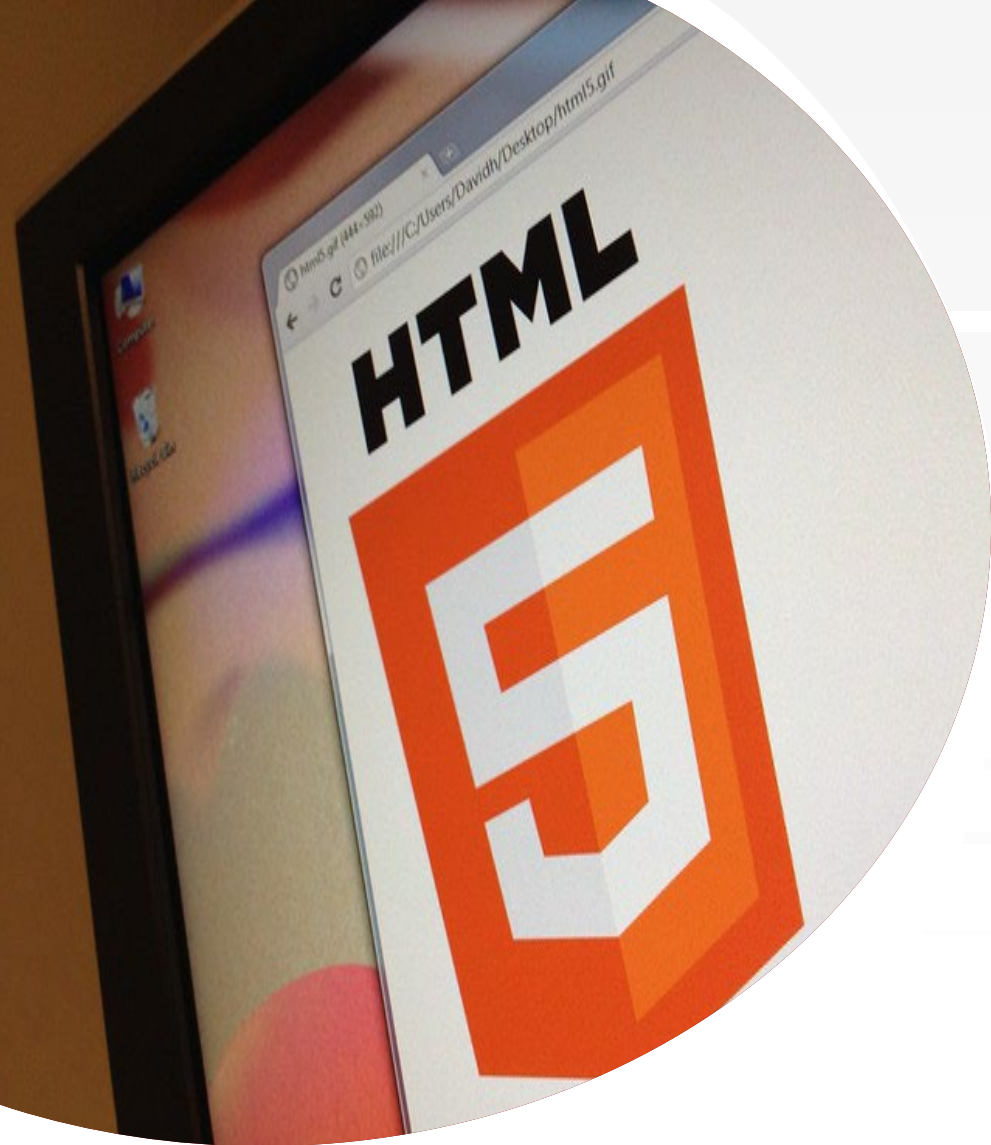


THE WORLD WIDE WEB

- History
- Components of the Web
 - Uniform Resource Identifier (URI/URL)
 - Hypertext Markup Language (HTML)
 - Hypertext Transfer Protocol (HTTP)
- Web browser history and the “browser wars.”
- Standardization and the W3C
- Client-side web development vs. server-side

CLIENT AND SERVER WEB MODEL





HTML5 AND THE SEMANTIC WEB

- What is HTML?
- History and goals of HTML5
- Basic HTML5 syntax and document structure
- Elements and attributes
 - Nesting of elements
- Character entities
- Images and links
- Semantic markup
 - What is it and why should we use it?
- Headers and footers
- Navigation
- Figures & captions
- Sections and articles
- Should we still use `<div>`?

HTML5

Three Main Goals



1. Improve the Native Web

- HTML5 adds syntactic features to the Web that could previously only be accomplished with plug-ins. Examples: the <audio>, <video>, and <canvas> elements.
- Enables a much richer, desktop-like experience (utilizing JavaScript).

2. Do More with Less Code

- HTML5 simplifies common design patterns (and more) by creating standardized ways to accomplish them in HTML alone. Examples: placeholder text in forms, autofocusing on a particular input element, client-side validation of form input, date and time pickers, etc.

3. Realize the Semantic Web

- A long-held dream of Tim Berners-Lee, the semantic web refers to content that's not only readable by humans but also understood by machines.

Summary

- The Internet:
 - originated in 1969 as the ARPANET
 - was designed to be fault-tolerant, simple and scalable
 - operates on a system of layered networking protocols (TCP/IP)
 - is decentralized and low cost
 - supports both client-server and peer-to-peer services
- The World Wide Web:
 - developed in 1989 by Tim Berners Lee (URL, HTTP, HTML)
 - runs at the application layer over the Internet
 - exploded in popularity in the mid-1990's due to advent of GUI web browsers (Mosaic, Netscape, IE)
- HTML5:
 - first released in 2008, with major update and W3C endorsement in 2014
 - latest and by far most popular revision of the HTML standard, replaces earlier efforts such as XHTML, DHTML, etc.
 - introduces the concept of semantic web
- The Internet and the World Wide Web are not the same thing!
 - Internet: 1969 (ARPANET)
 - Web: 1989 (Tim Berners Lee)

► **Additional Materials for Self-study**

Fundamentals of Web Development

Third Edition by Randy Connolly and Ricardo Hoar



Chapter 1

Introduction to Web Development

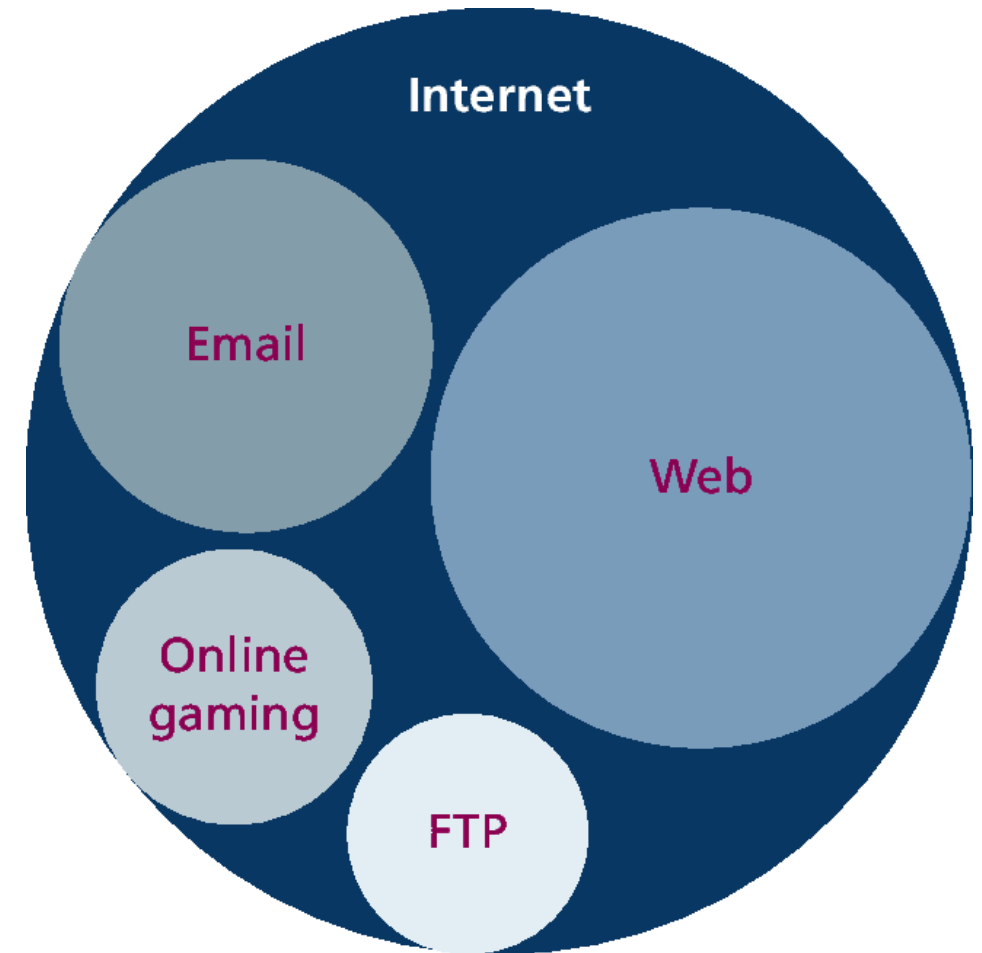
In this chapter you will learn . . .

- **About web development in general**
- **The history of the Internet and World Wide Web**
- **Fundamental concepts that form the foundation of the Internet**
- **About the hardware and software that support the Internet**
- **The range of careers and companies in web development**



A Short History of the Internet

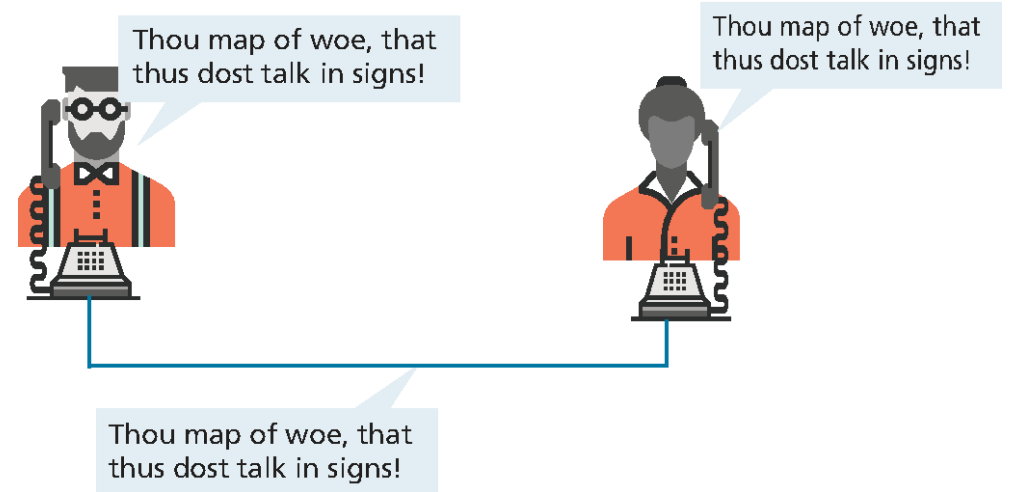
- Know the difference between “Internet” and “WWW”
- While this book is focused on the web, part of this chapter is also devoted to a broad understanding of that larger circle labeled the “Internet.”
- **Protocols** define different kinds of interactions/services on the Internet



Circuit Switched Networks

Circuit switching in early networking

- In the past, telephone calls were routed through operators who **physically connected** the caller and the receiver by connecting a wire to a switchboard to complete a circuit.
- Inefficient use of **bandwidth**
- Difficult to scale

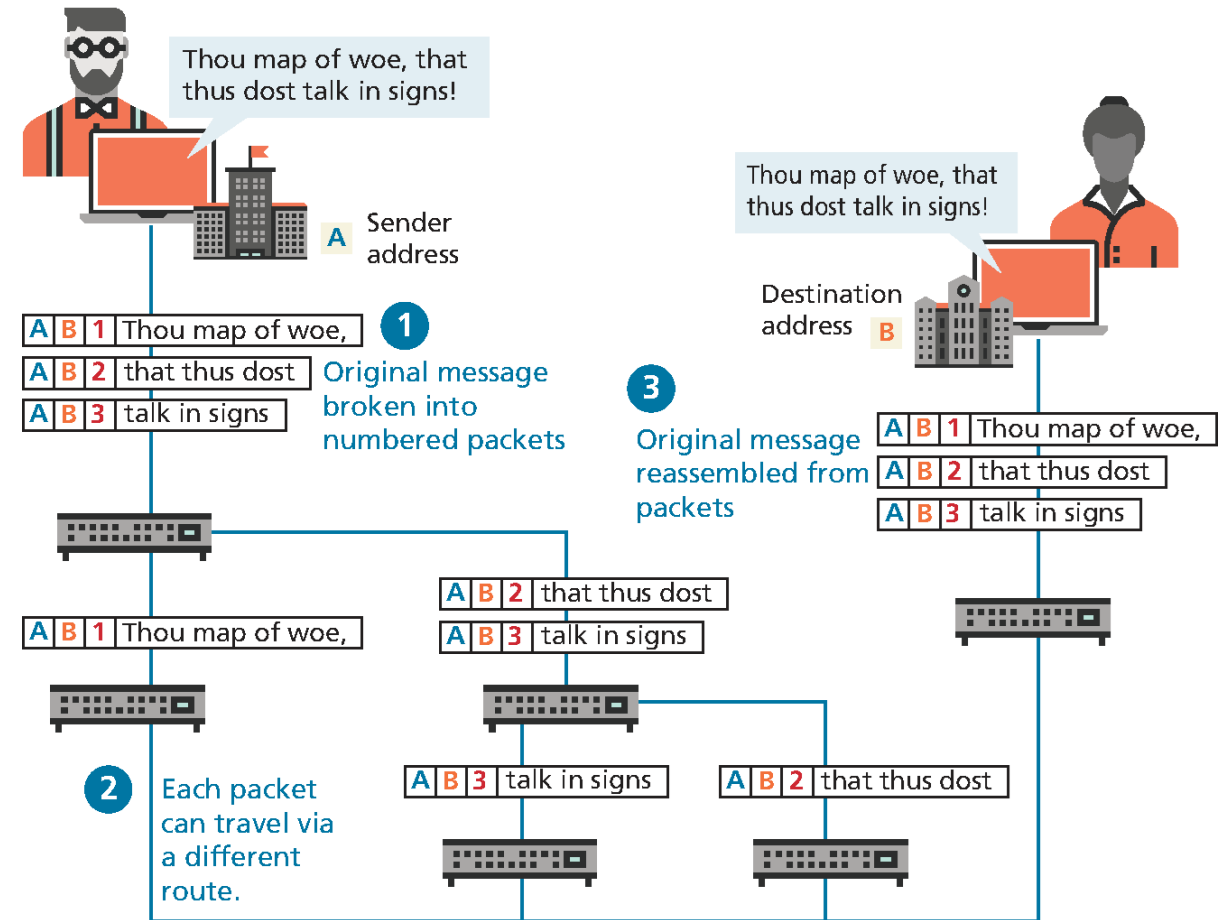


Packet Switched Networks

Packet switching came later, does not require a continuous connection

- 1960s ARPANET
- 1974 X.25
- 1979 USENET
- 1981 TCP/IP was introduced to unify disparate networks

On January 1, 1983, TCP/IP was adopted across all of ARPANET



The Birth of the Web

Sr. Tim Berners-Lee publishes the main features of the web we know today on 1992.

- A Uniform Resource Locator (**URL**) to uniquely identify a resource on the WWW.
- The Hypertext Transfer Protocol (**HTTP**) to describe how requests and responses operate
- A software program (**web server software**) that can respond to HTTP requests.
- Hypertext Markup Language (**HTML**) to publish documents.
- A program (**a browser**) that can make HTTP requests to URLs and that can display the HTML it receives.



Web Applications in Comparison to Desktop Applications (1 of 2)

Advantages

- They can be accessed from any Internet-enabled computer.
- They can be used with different operating systems and browser applications.
- They are easier to roll out program updates since only software on the server needs to be updated as opposed to every computer in the organization using the software.
- They have a centralized storage on the server, which means fewer security concerns about local storage (which is important for sensitive information such as health care data).



Web Applications in Comparison to Desktop Applications (2 of 2)

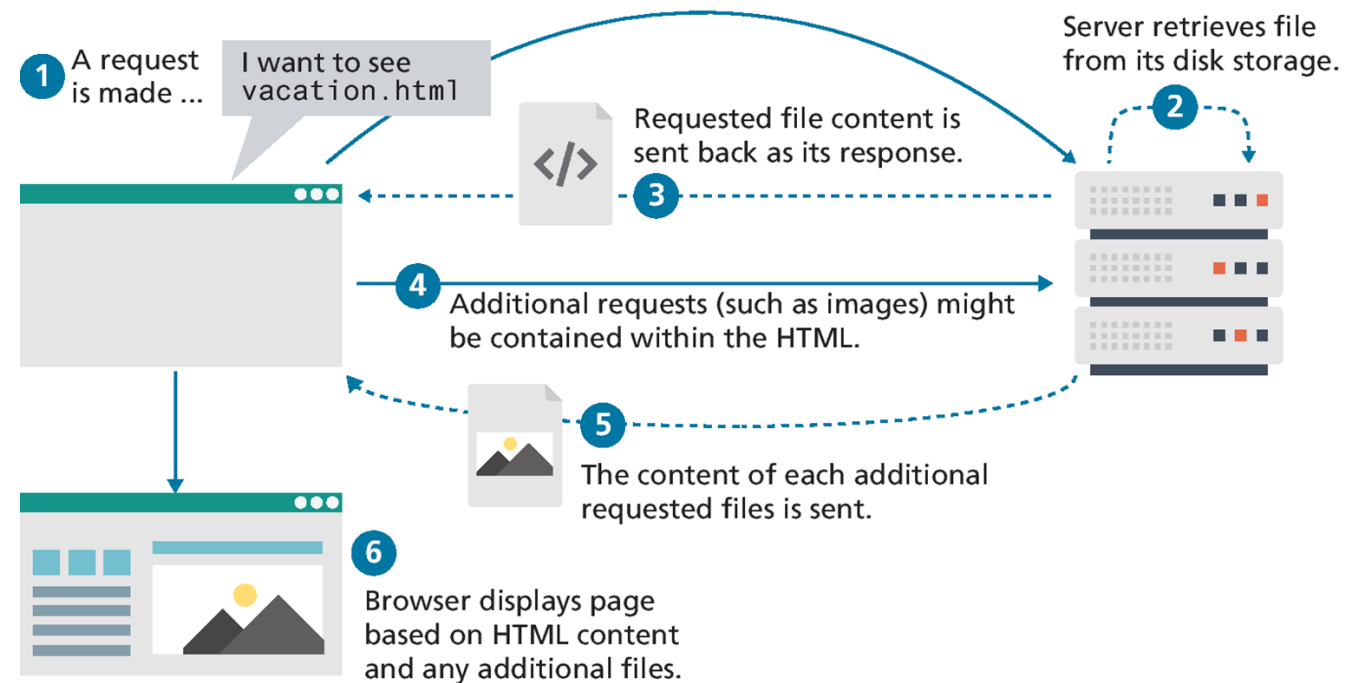
Disadvantages

- Requirement to have an active Internet connection
- Security concerns about sensitive private data being transmitted over the Internet.
- Concerns over the storage, licensing, and use of uploaded data.
- Problems with certain websites not having an identical appearance across all browsers.
- Restrictions on software from being installed and hardware from being accessed (like Adobe Flash on iOS).
- additional plugins might interfere with JavaScript, cookies, or advertisements.



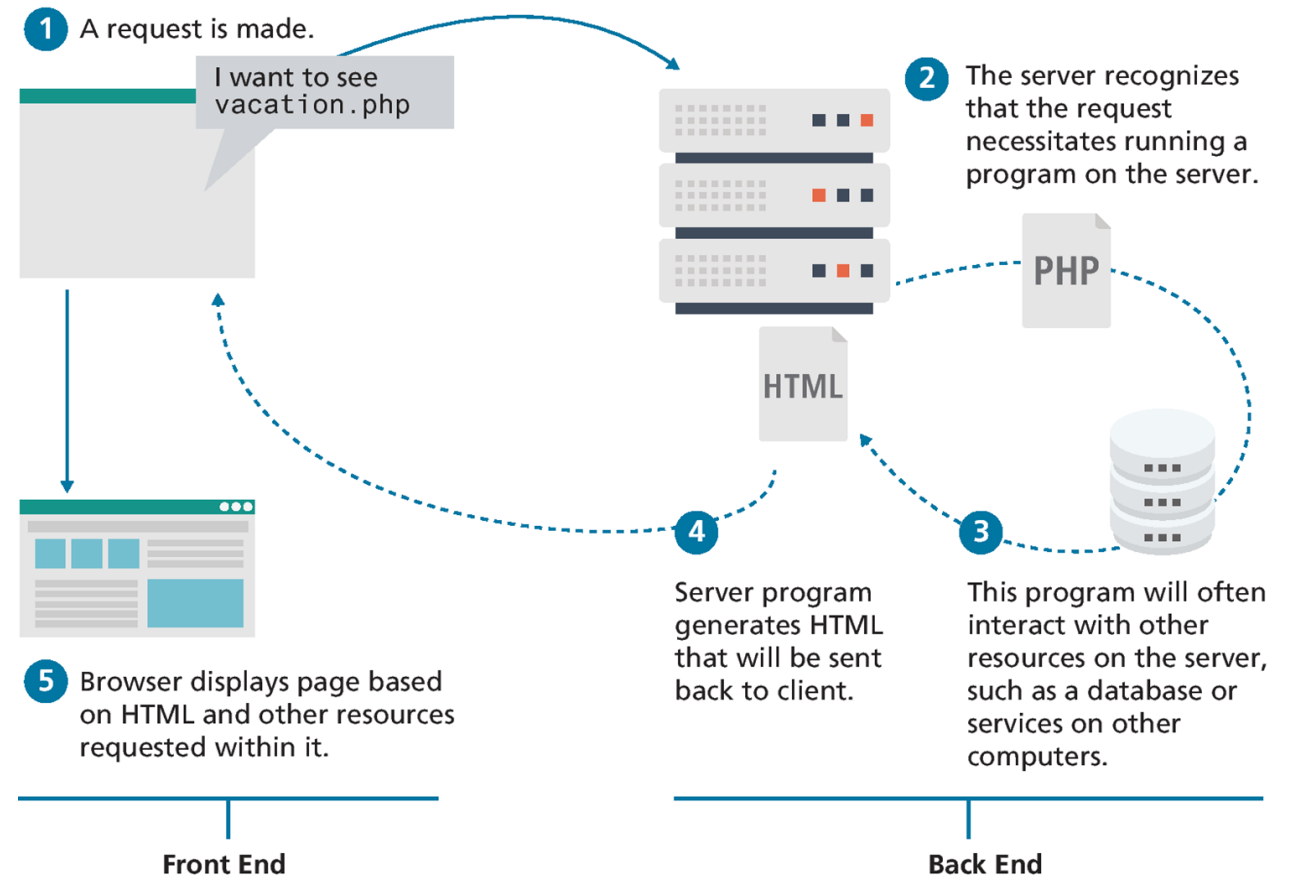
From Static to Dynamic (and Back to Static)

In the earliest days of the web, users could read the pages of a **static website**



From Static to Dynamic (and Back to Static) ii

Later, programs running on web servers let websites generate content dynamically. This type of website is called a **dynamic server-side website**

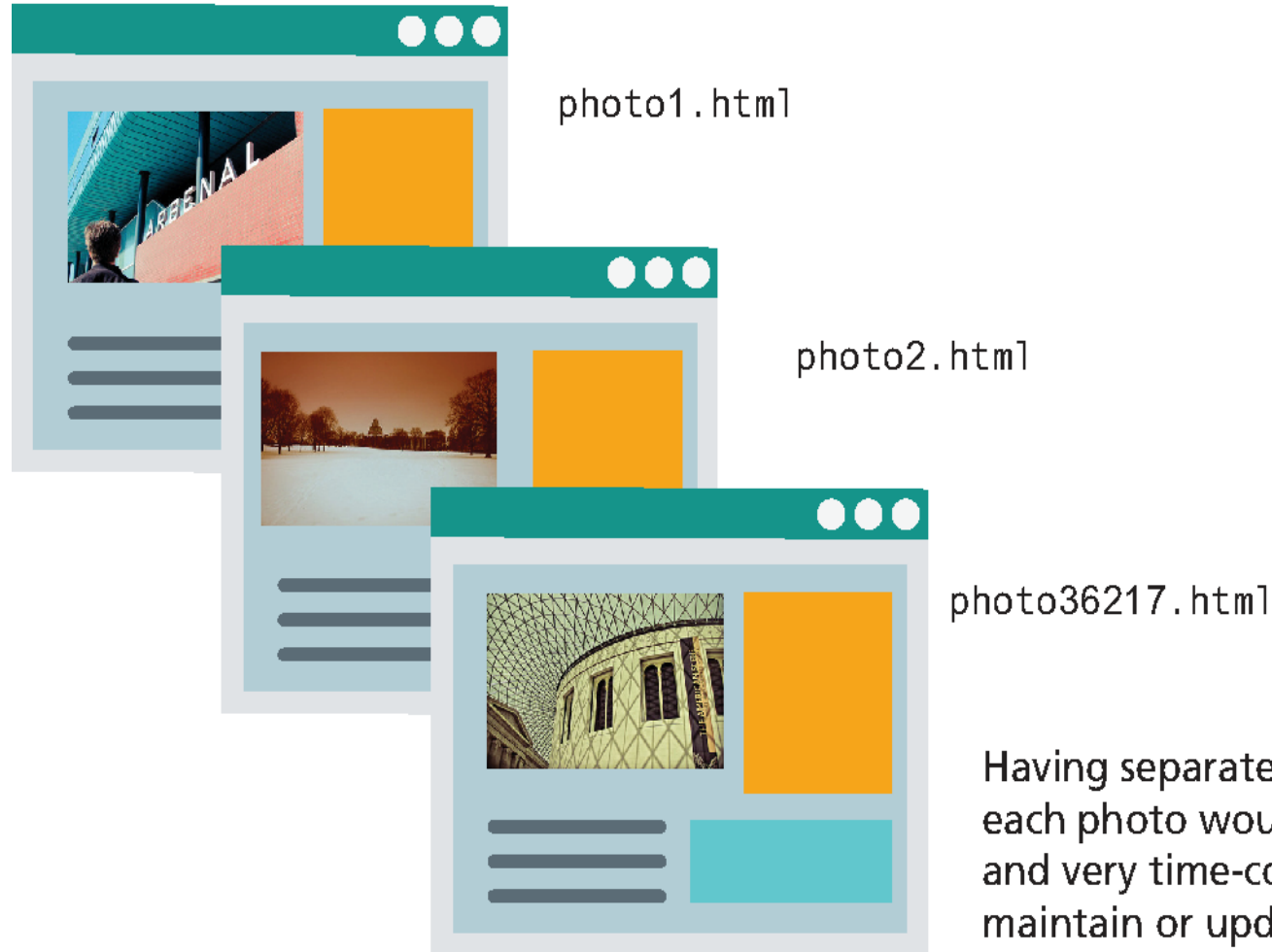


Web 2.0

- **Web 2.0** referred to an interactive experience where users could contribute and consume web content, thus creating a more user-driven web experience.
- For software developers, Web 2.0 also referred to a change in the paradigm. Programming logic, which previously existed only on the server, began to migrate more and more to the browser, which required learning **JavaScript**



Why are programs needed? (1 of 2)

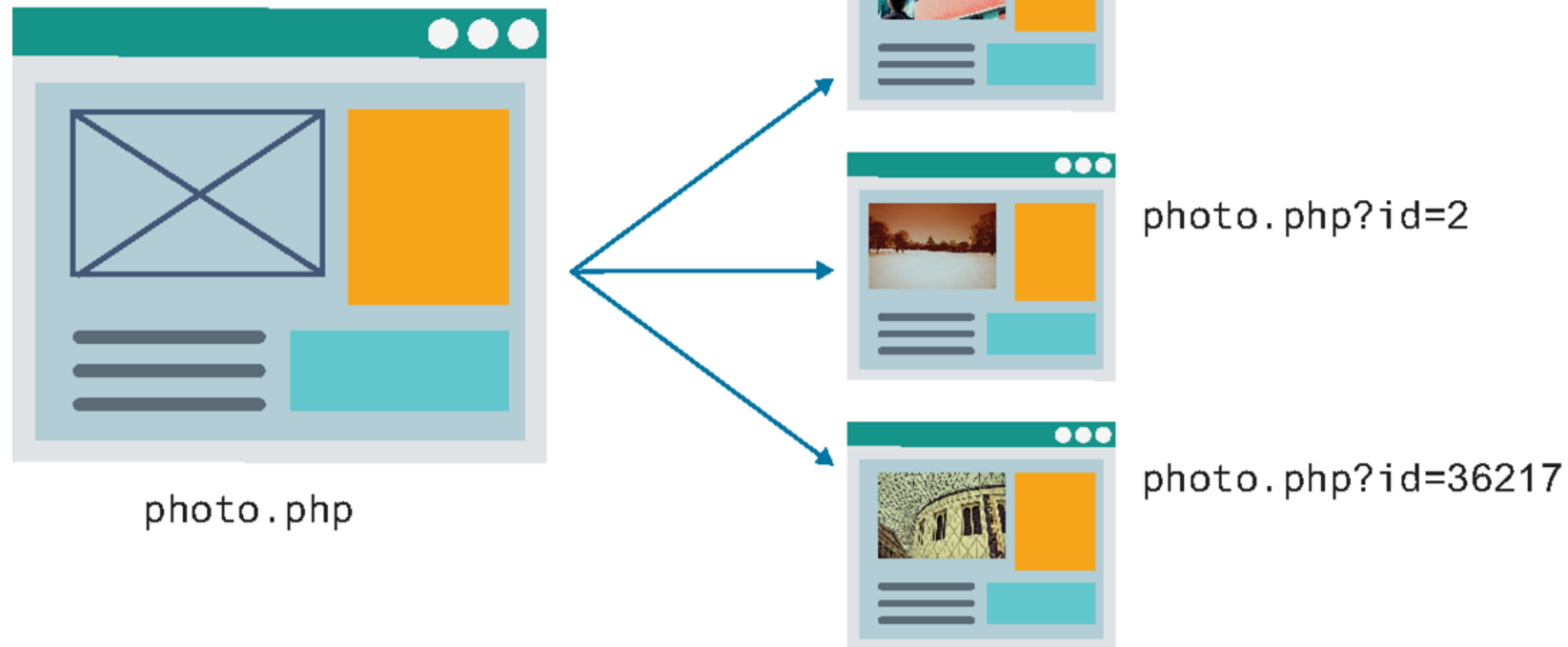


Having separate HTML pages for each photo would be impractical and very time-consuming to maintain or update the design.



Why are programs needed? (2 of 2)

Instead, using a single program, we can display any of our photos. We will only need to update a single program when we want to revise the design.



The evolution continues

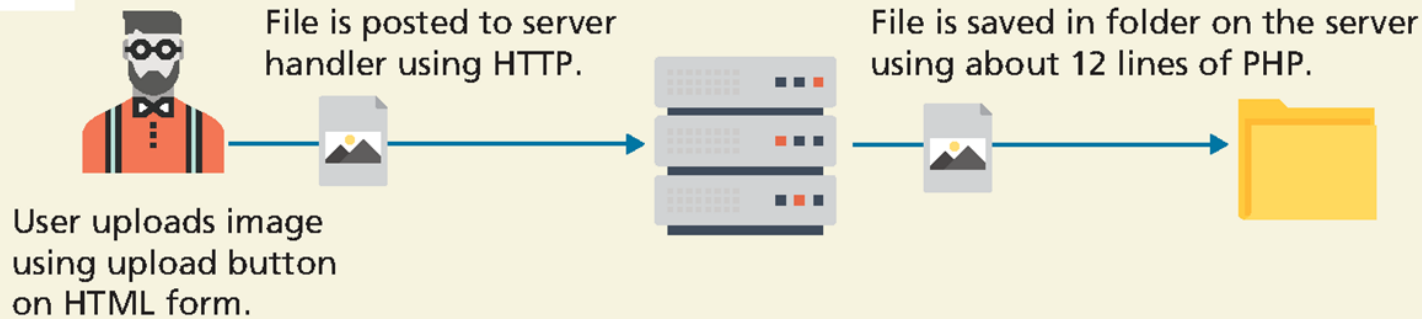
Web development today is thus more complicated than it was when the first edition of this textbook was written in 2012–2013.

- Early chapters on **HTML** and **CSS** teach layout and structural foundations.
- **JavaScript** chapters focus on the fundamentals of the language and its usage within the browser.
- While back-ends are thinner than they once were, **server-side technologies** are still essential.
- Databases, state management, and authentication are all covered.
- Management, security and configuration round out the advanced topics.
- The one constant in the history of web development has been change

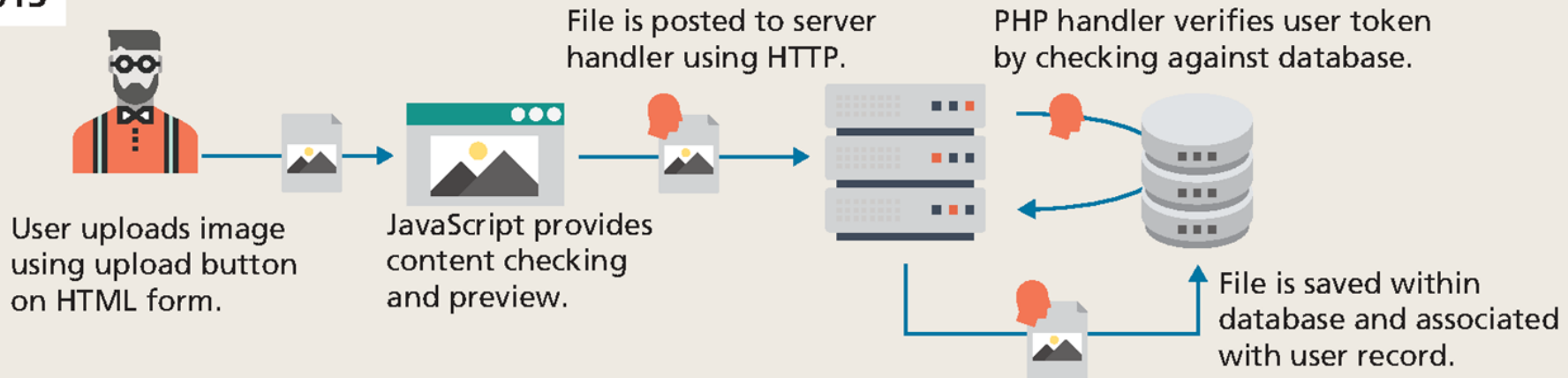


Evolving complexity example. File upload

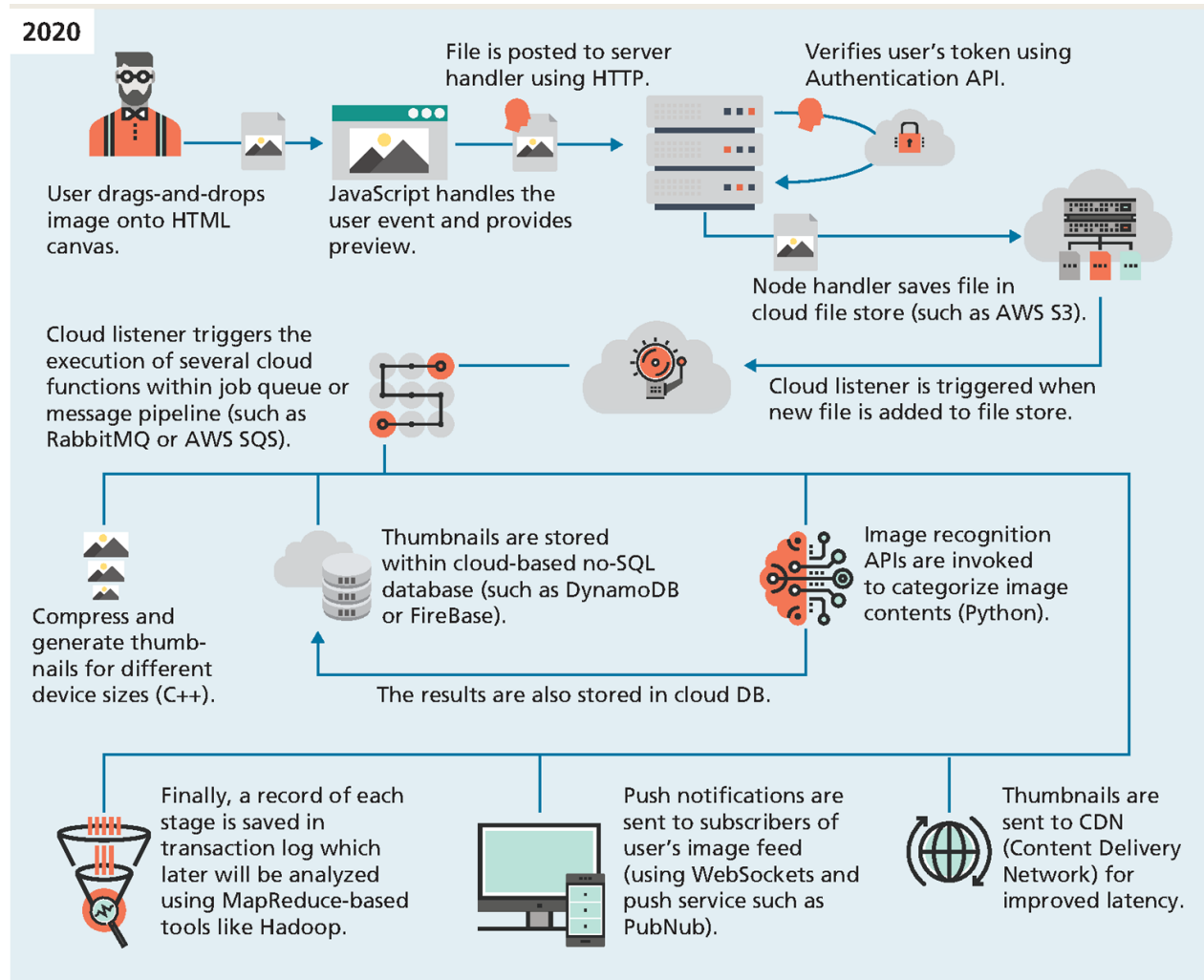
2006



2013



Evolving complexity example (cont.)



The Client-Server Model

Client machines are the desktops, laptops, smart phones, and tablets you see everywhere.

Broad range of specifications regarding

- operating system,
- processing speed,
- screen size,
- available memory, and
- storage

Server machines hosts web applications, stores user and program data, and performs security authorization tasks

Powerful machines to handle high traffic and bandwidth.

The essential characteristic of a server is that it is listening for requests, and upon getting one, responds with a message.



Server Types

- Web servers.
- Application servers.
- Database servers.
- Mail servers.
- Media servers.
- Authentication servers.



Real World Server Installations

Not one server, but a cluster of multiple machines working together.

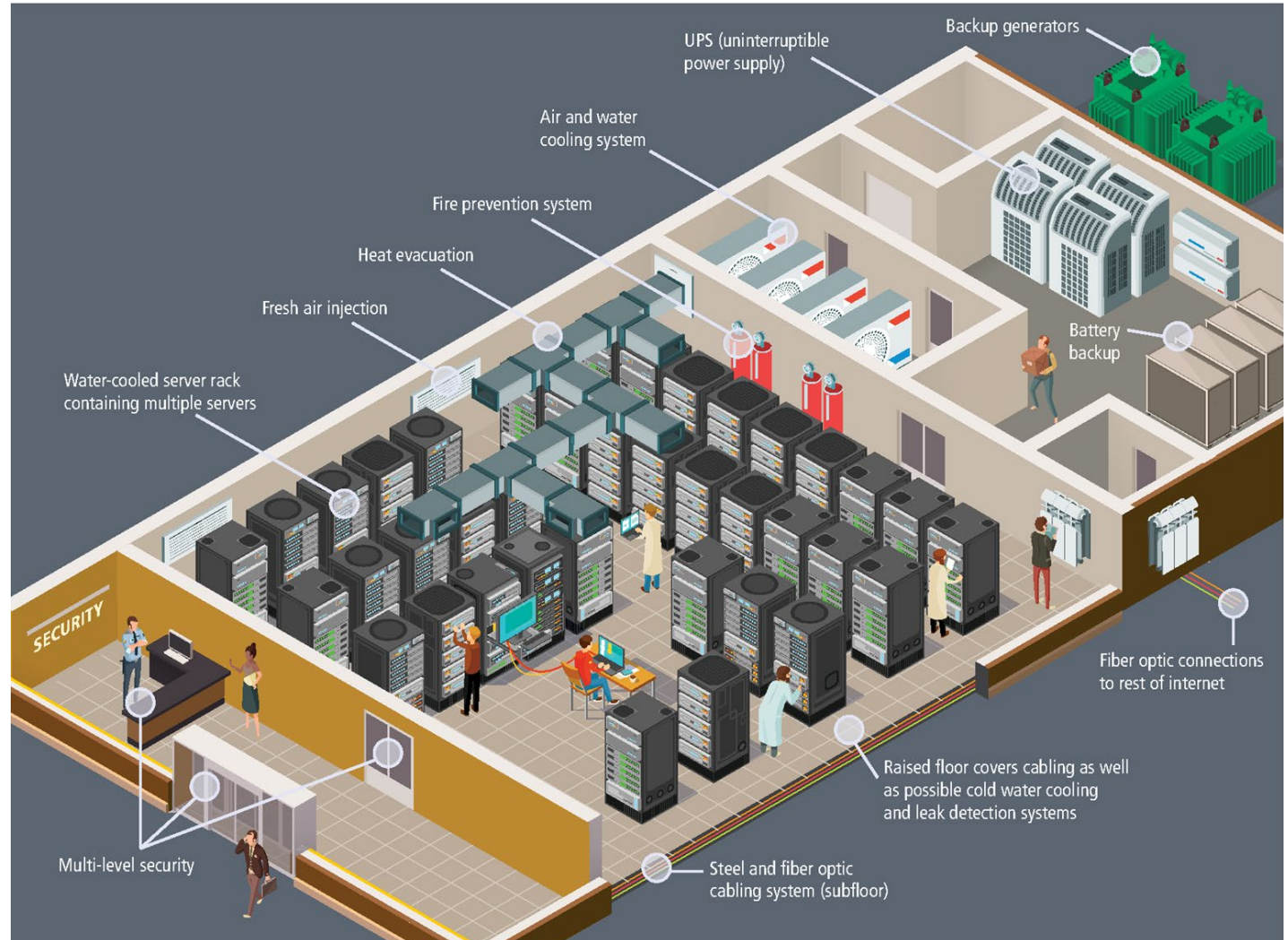
- Server Farm
- Load Balancers
- Failover Redundancy
- Server Racks
- Data Centers
- Cloud Services



Hypothetical data center

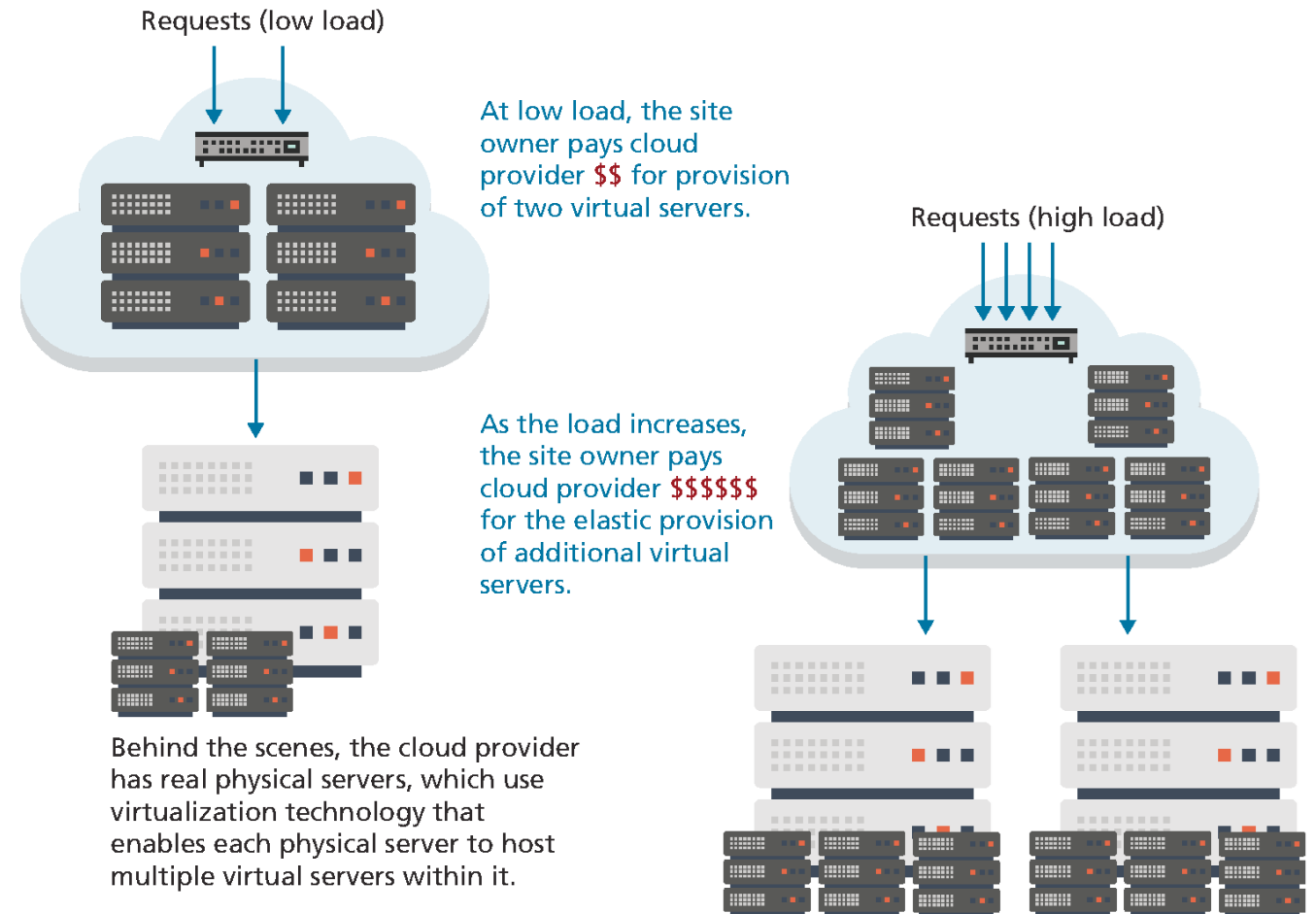
Many additional considerations can be handled at a data center including:

- Fire suppression,
- Biometric security,
- Failover data
- Redundant power and more!



Cloud Servers

Instead of spending too much or spending too little to handle peak loads, cloud providers offer **elastic provisioning** of virtual servers, which scales costs and hardware to the demand



Where Is the Internet?

It is quite common for the Internet to be visually represented as a cloud

Actually implemented via millions of miles of copper wires and fiber optic cables connecting millions of server computers and other networked devices!



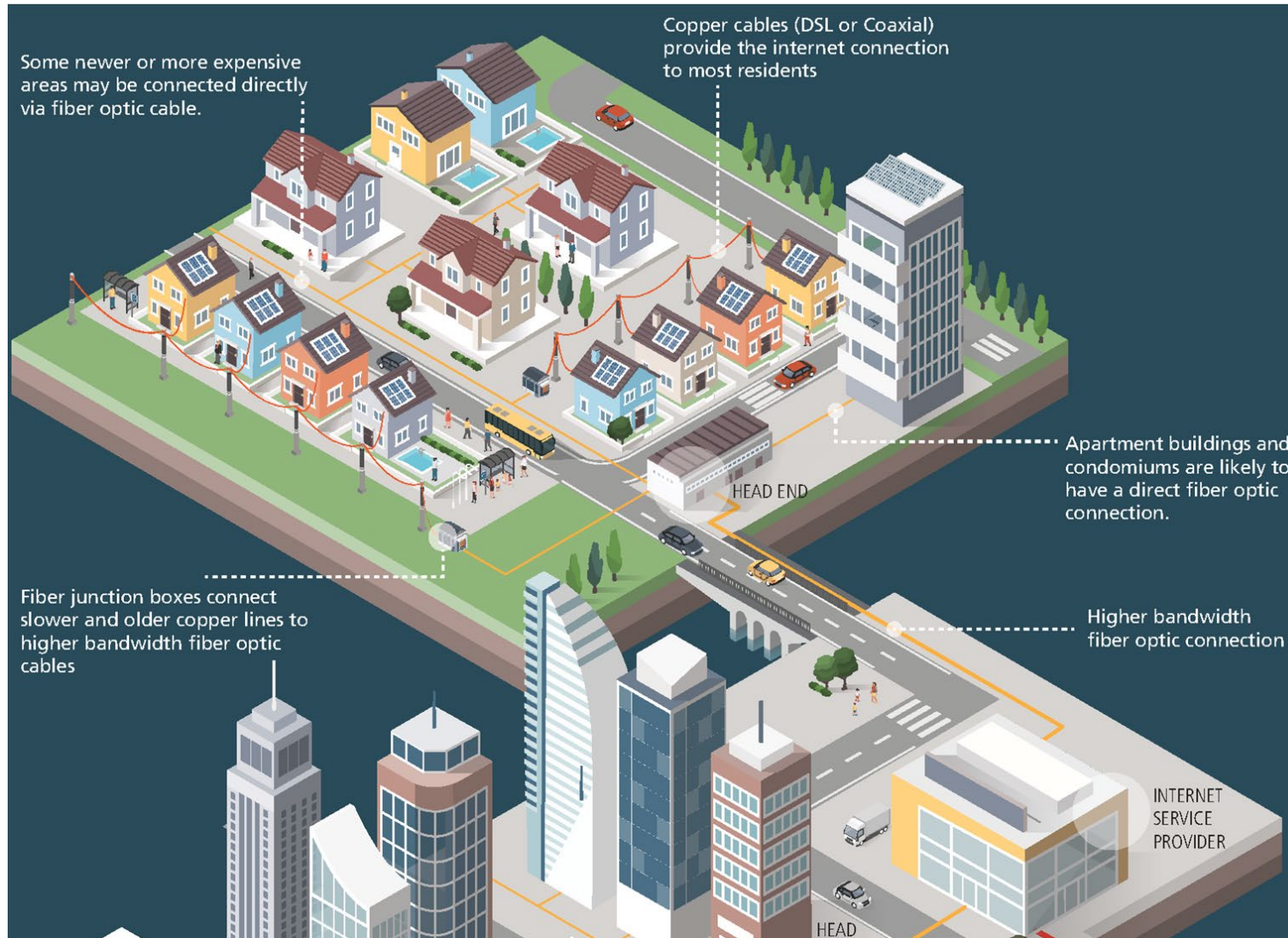
From the Computer to Outside the Home

The **broadband modem** is a bridge between the network hardware outside the house and the network hardware inside the house. These devices are often supplied by the ISP.

The **wireless router** is perhaps the most visible manifestation of the Internet in one's home. At its simplest, a router is a hardware device that forwards data packets from one network to another network.

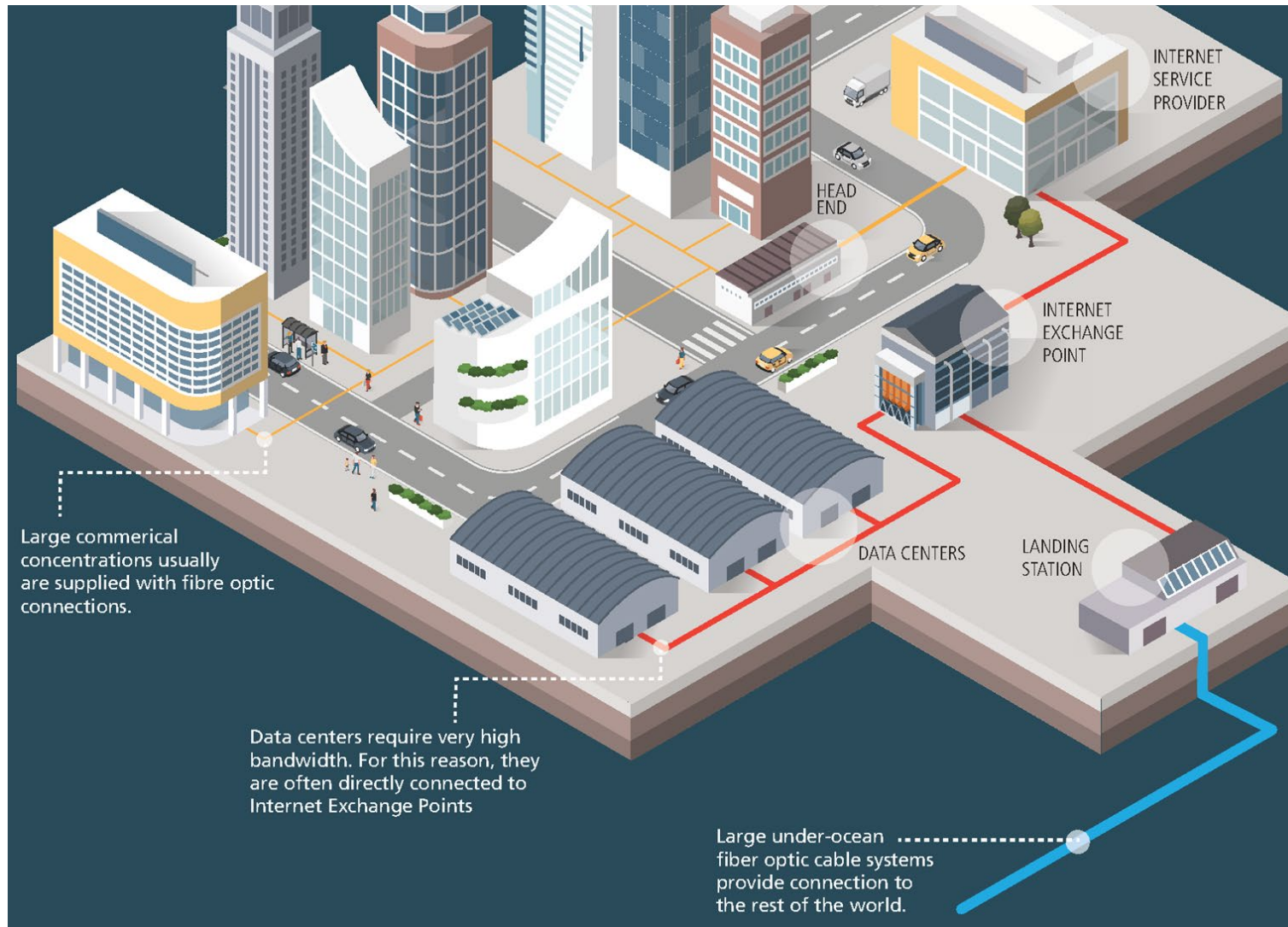


From the Home to the Ocean's Edge

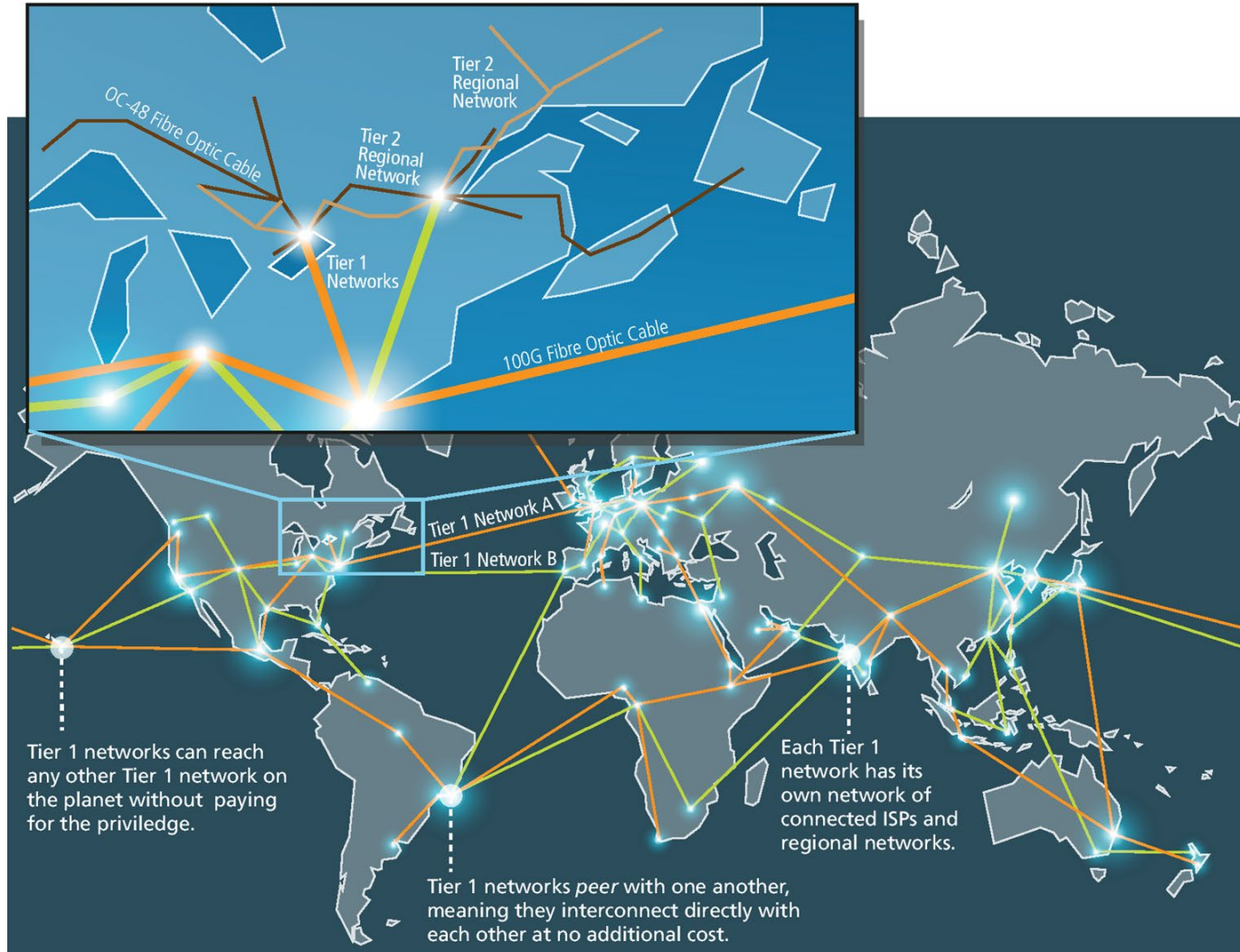


From the Home to the Ocean's Edge

(2)



How the Internet Is Organized Today

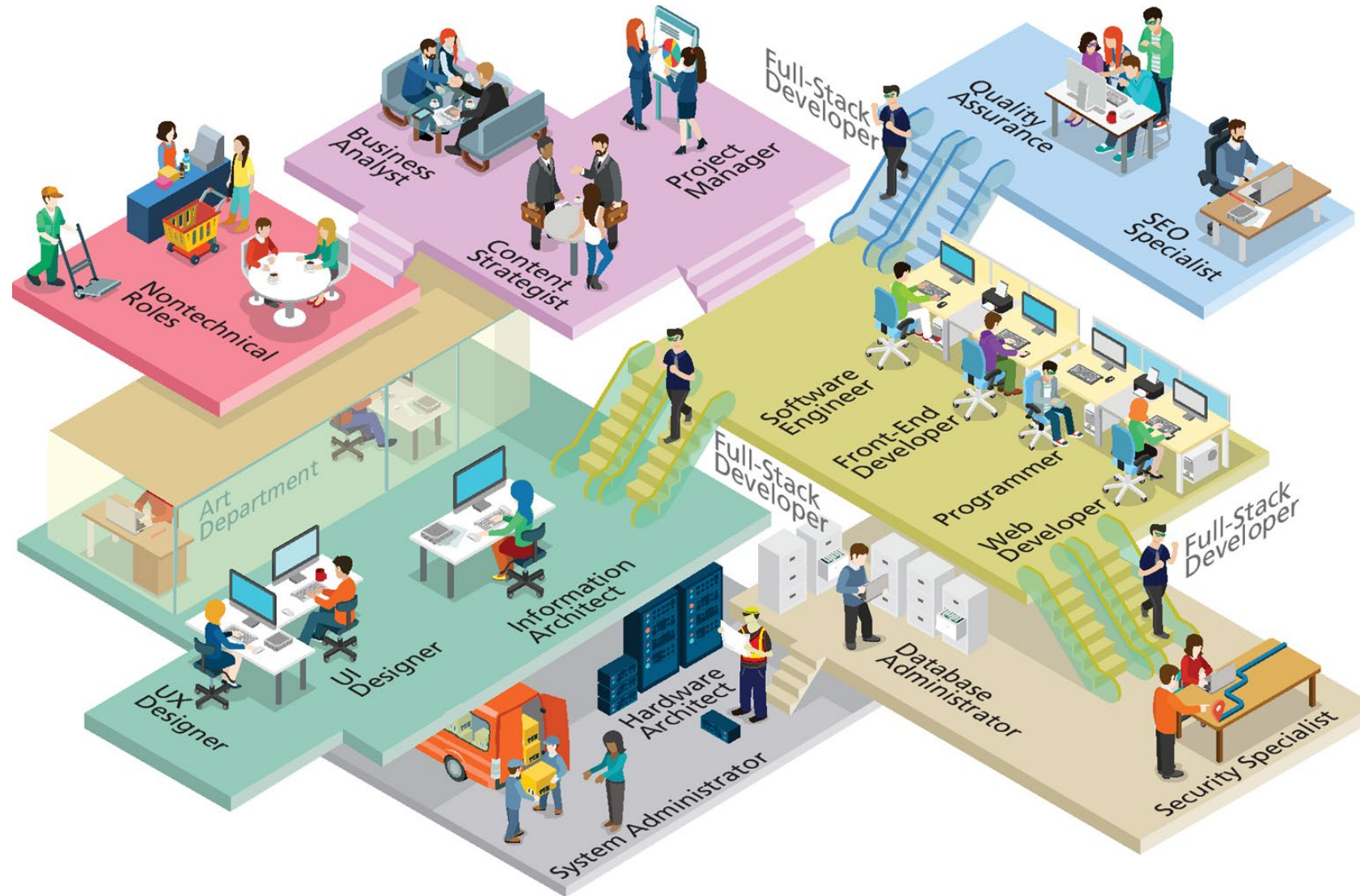


When someone talks about the “Internet Backbone” they are talking about Tier 1 networks.

Tier 1 Networks make use of very fast fiber optic cable.

Regional networks have traditionally used less speedy infrastructure, though this is rapidly changing as prices of optical hardware decreases.

Working in Web Development



Roles in Web Development

- Hardware Architect/Network Architect/Systems Engineer
- System Administrator
- Database Administrator/Data Architect
- Security Specialist/Consultant/Expert
- Developer/Programmer
- Front-End Developer/UX Developer
- Software Engineer
- UX Designer/UI Designer/Information Architect
- Tester/Quality Assurance
- SEO Specialist
- Content Strategists/Marketing Technologist
- Project Manager/Product Manager
- Business Analyst
- Nontechnical Roles
- Full-stack developer



Web Development Companies

- Hosting Companies
- Design Companies
- Website Solution Companies
- Vertically Integrated Companies
- Start-Up Companies
- Internal Web Development



Key Terms

application server
authentication server
back end
bandwidth
broadband modem
circuit switching
client
client-server model
Content Delivery
Networks (CDN)
Content Delivery
Networks (CDN)
data center
database server
DevOps (Development
and Operations)
dynamic server-side
website
elastic provisioning
failover redundancy
fiber optic cable
front end
full-stack developer
intranet

latency
load balancers
mail server
media server
Mosaic
Netscape Navigator
Network Access Points
(NAP)
next-hop routing
packet
packet switching
peer-to-peer model
request
protocols
Request for Comments
(RFC)
request-response loop
response
router
routing table
semantic web
server
server farm
server racks

user experience
virtual server
webmaster
web servers
Web 2.0
World Wide Web
Consortium (W3C)



Fundamentals of Web Development

Third Edition by Randy Connolly and Ricardo Hoar



Chapter 2

How the Web Works

In this chapter you will learn . . .

- The fundamental protocols that make the web possible
- How the domain name system works
- Why HTTP is more than just a four-letter abbreviation
- How browsers and servers work to exchange and interpret HTML

Internet protocols

- A **protocol** is a set of rules that partners use when they communicate.
- TCP/IP, from Chapter 1, is an essential internet protocol!
- These protocols have been implemented in every operating system and make fast web development possible. If web developers had to keep track of packet routing, transmission details, domain resolution, checksums, and more, it would be hard to get around to the matter of actually building websites.

Starting Note

NOTE

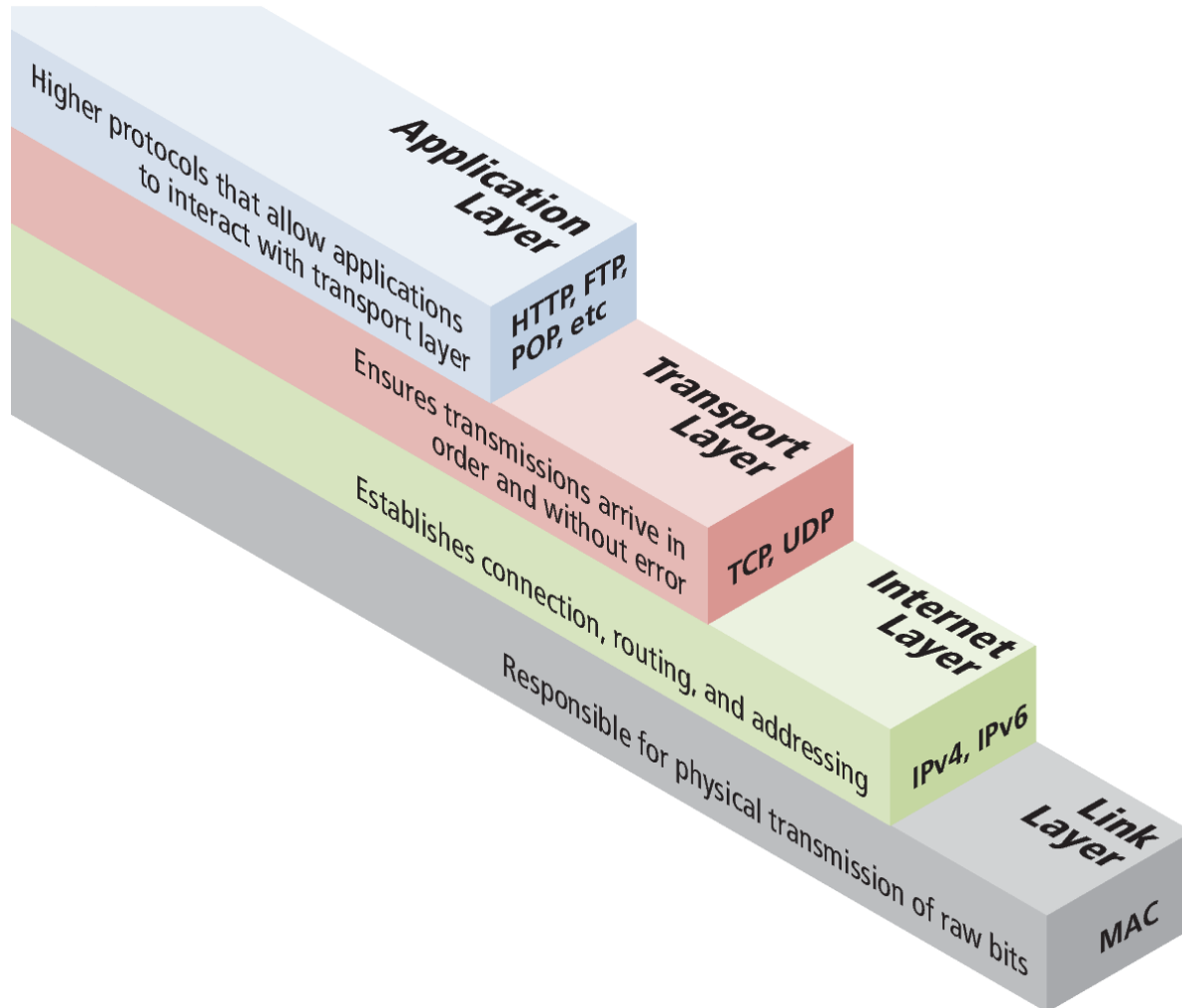
Knowledge of how the web works, from low-level protocol to high-level JavaScript library, creates better web developers, which is why we start with some fundamental concepts in these early chapters.

There is a trend in web development to encourage web developers and designers to embrace this blending of roles as part of a holistic *DevOps* approach, which we describe in Chapter 17.

This means even if you're hired primarily to style CSS, you may need to know about HTML, IP addresses, domain names, web servers, browsers and more. Thankfully, you can always come back and revisit this material later when it's referenced again.



A Layered Architecture



- The TCP/IP Internet protocols were originally abstracted as a four-layer stack
- Later abstractions subdivide it further into five or seven layers
- Since we focus on the top layer, we will use the earliest and simplest **four-layer network model**.

Link Layer

- The **link layer** is the lowest layer, responsible for both the physical transmission of data across media and establishing logical links.
- It handles issues like packet creation, transmission, reception, error detection, collisions, line sharing, and more.
- One term that is sometimes used in the Internet context is that of **MAC** (media access control) **addresses**.

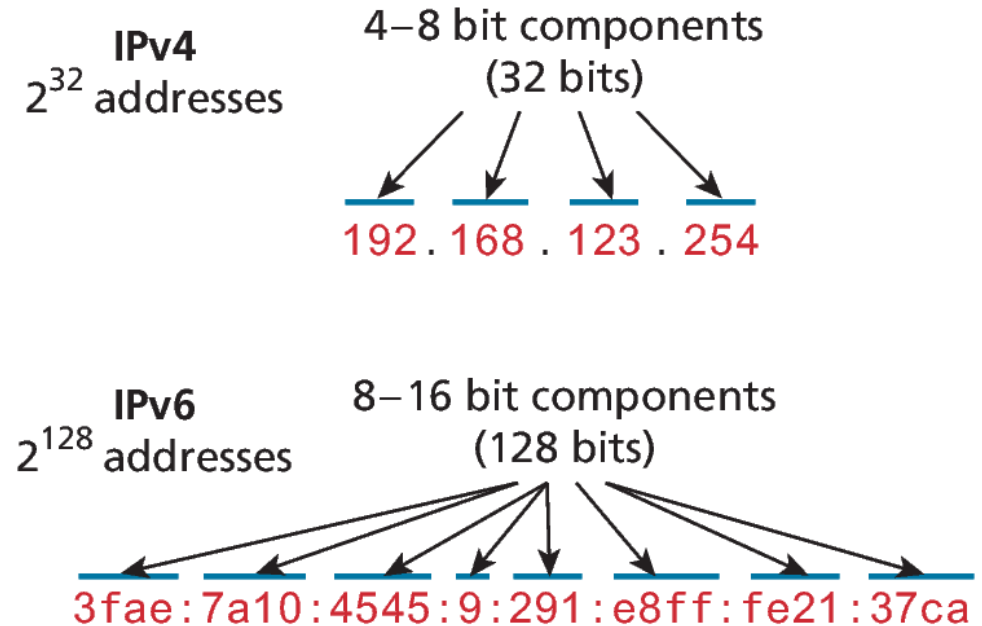
Internet Layer

- The **Internet layer** (sometimes also called the IP Layer) routes packets between communication partners across networks.
- It provides “best effort” communication. It sends out a message to its destination but expects no reply and provides no guarantee the message will arrive intact, or at all.
- The Internet uses the **Internet Protocol (IP) addresses**, which are numeric codes that uniquely identify destinations on the Internet.
- Every device connected to the Internet has such an **IP address**.

IP Addresses (cont)

There are two types of IP addresses: IPv4 and IPv6.

- In **IPv4**, four 8-bit integers separated by . encode the address.
- **IPv6** uses eight 16-bit integers and has over a billion billion times the number in IPv4



Port Address Translation (PAT)

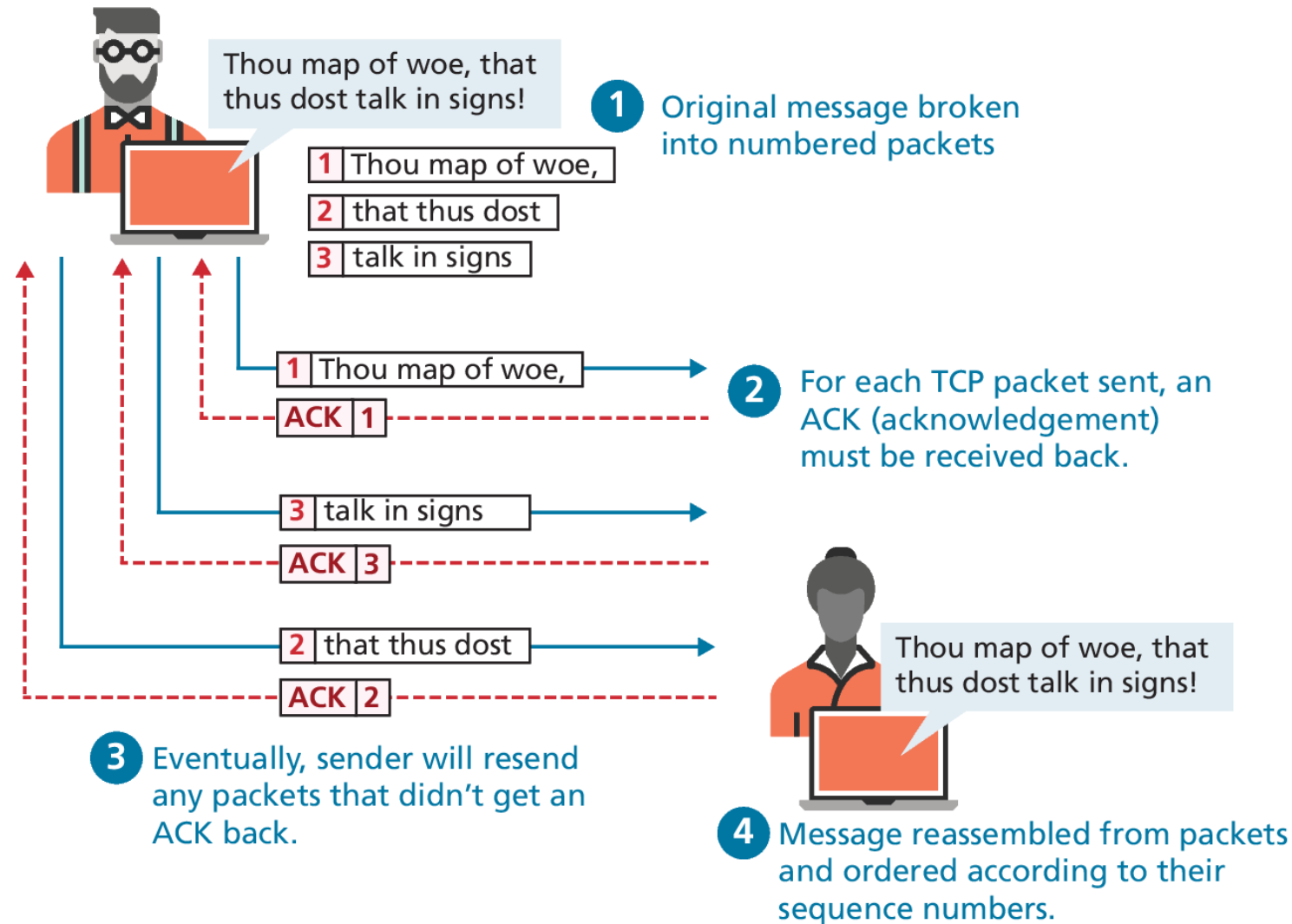
The IPv4 address space was depleted in 2011, but the number of computers connected to the Internet continued to grow.

- **Port Address Translation (PAT)**, allows multiple, unrelated networks to make use of the same IP address
- When you join a wireless network in a coffee shop, home, office or university, it is quite likely you are making use of PAT.
- For future growth, IPv6 will be necessary.

Transport Layer

- The **transport layer** ensures transmissions arrive in order and without error.
- First, the data is broken into **packets** formatted according to the **Transmission Control Protocol (TCP)**.
 - Each data packet has a header that includes a sequence number, so the receiver can put the original message back in order
 - Each packet acknowledges its successful arrival back to the sender (ACK).
 - In the event of a lost packet (since no ACK arrived for that packet the packet will be retransmitted.
- This means you have **a *guarantee*** that messages sent will arrive and will be in order.

Transport Layer (example)



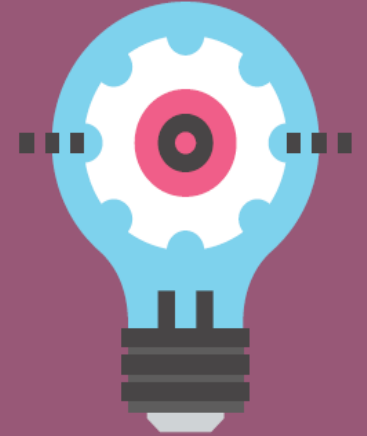
User Datagram Protocol (UDP)

PRO TIP

Sometimes we do not want guaranteed transmission of packets.

Consider a live multicast of a soccer game, for example. Millions of subscribers may be streaming the game, and the broadcaster can't afford to track and retransmit every lost packet. A small loss of data in the feed is acceptable, and the customers will still see the game.

An Internet protocol called **User Datagram Protocol (UDP)** is used in these scenarios in lieu of TCP. Other examples of UDP services include Voice Over IP (VoIP), many online games, and Domain Name System (DNS).



Application Layer

- The **application layer** is the level of protocols familiar to most web developers.
- Application layer protocols implement process-to-process communication.
- There are many application layer protocols. A few that are useful to web developers include:
 - **HTTP**. The Hypertext Transfer Protocol is used for web communication.
 - **SSH**. The Secure Shell Protocol allows remote command-line connections to servers.
 - **FTP**. The File Transfer Protocol is used for transferring files between computers.
 - **POP/IMAP/SMTP**. Email-related protocols for transferring and storing email.
 - **DNS**. The Domain Name System protocol used for resolving domain names to IP addresses.

Domain Name System

- As elegant as IP addresses may be, human beings do not enjoy having to recall long strings of numbers.
- Even as far back as the days of ARPANET, researchers assigned **domain names** to IP addresses
- In those early days, the number of Internet hosts was small, so a list of a domains and associated IP addresses could be downloaded as needed as a **hosts** file (see Pro Tip p51).
- As the number of computers on the Internet grew, this **hosts** file had to be replaced with a better, more scalable, and distributed system. This system is called the **Domain Name System (DNS)**

DNS Overview

- The DNS system maps resolves domain names to IP addresses.
- By separating the domain name of a server from its IP address, a site can move to a different host without changing its name.
- Since the entire request-response cycle can take less than a second, it is easy to forget that DNS requests are happening in all.
- The actual process is more complex (more on that later)

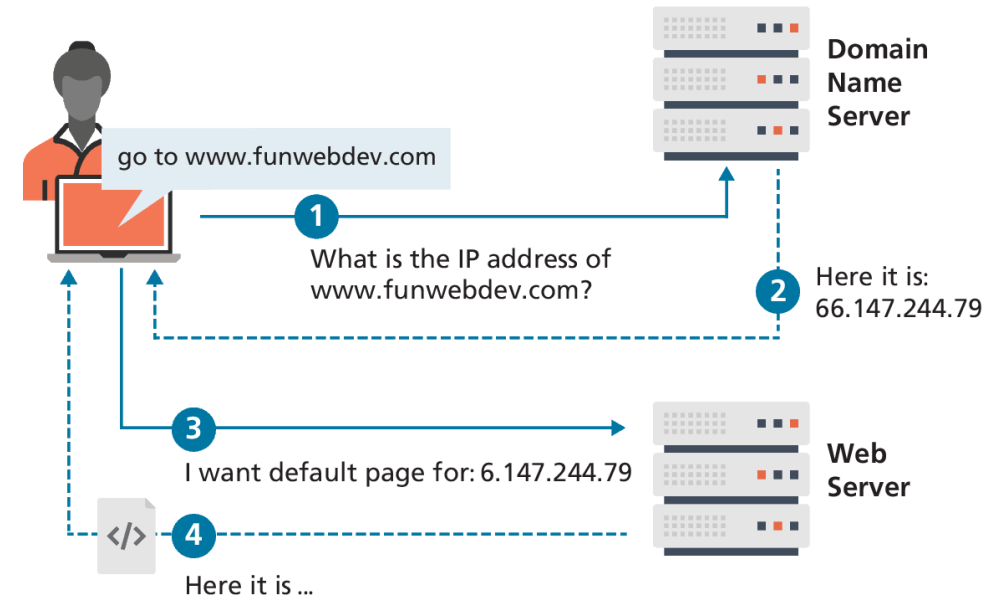
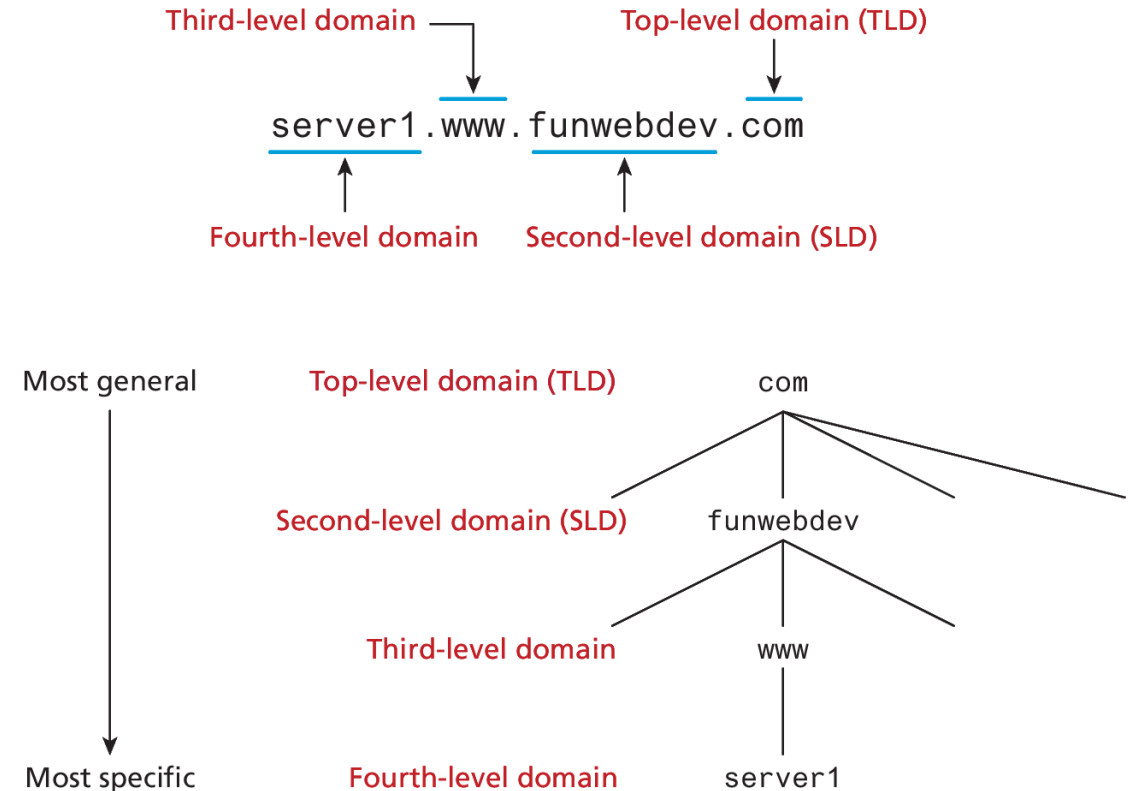


FIGURE 2.6 DNS overview (simplified)

Name Levels

- A domain name can be broken down into several parts, which describe a hierarchy.
- All domain names have at least a **top-level domain (TLD)** name and a **second-level domain (SLD)** name



Name Levels (Top Level)

The rightmost portion of the domain name (to the right of the rightmost period) is called **the top-level domain**. For the top level of a domain, we are limited to two broad categories, plus a third reserved for other use.

1. **Generic top-level domain (gTLD)**
2. **Country code top-level domain (ccTLD)**
3. **.arpa** (used for reverse DNS lookups)

Generic top-level domain (gTLD)

Generic top-level domains (gTLD) include the famous .com and .org. There are 3 subtypes of gTLD.

- **Unrestricted.** TLDs include **.com**, **.net**, **.org**, and **.info**.
- **Sponsored.** TLDs including **.gov**, **.mil**, **.edu**, and others
- **New.** Starting in June 2012, ICANN invited companies to launch new TLDs in order to provide more choice. Since then over 1000 new TLD have been created including .art, .cash, .cool, .jobs, .tax and so on

Country code top-level domain

Country code top-level domain (ccTLD) are under the control of the countries which they represent, which is why each is administered differently.

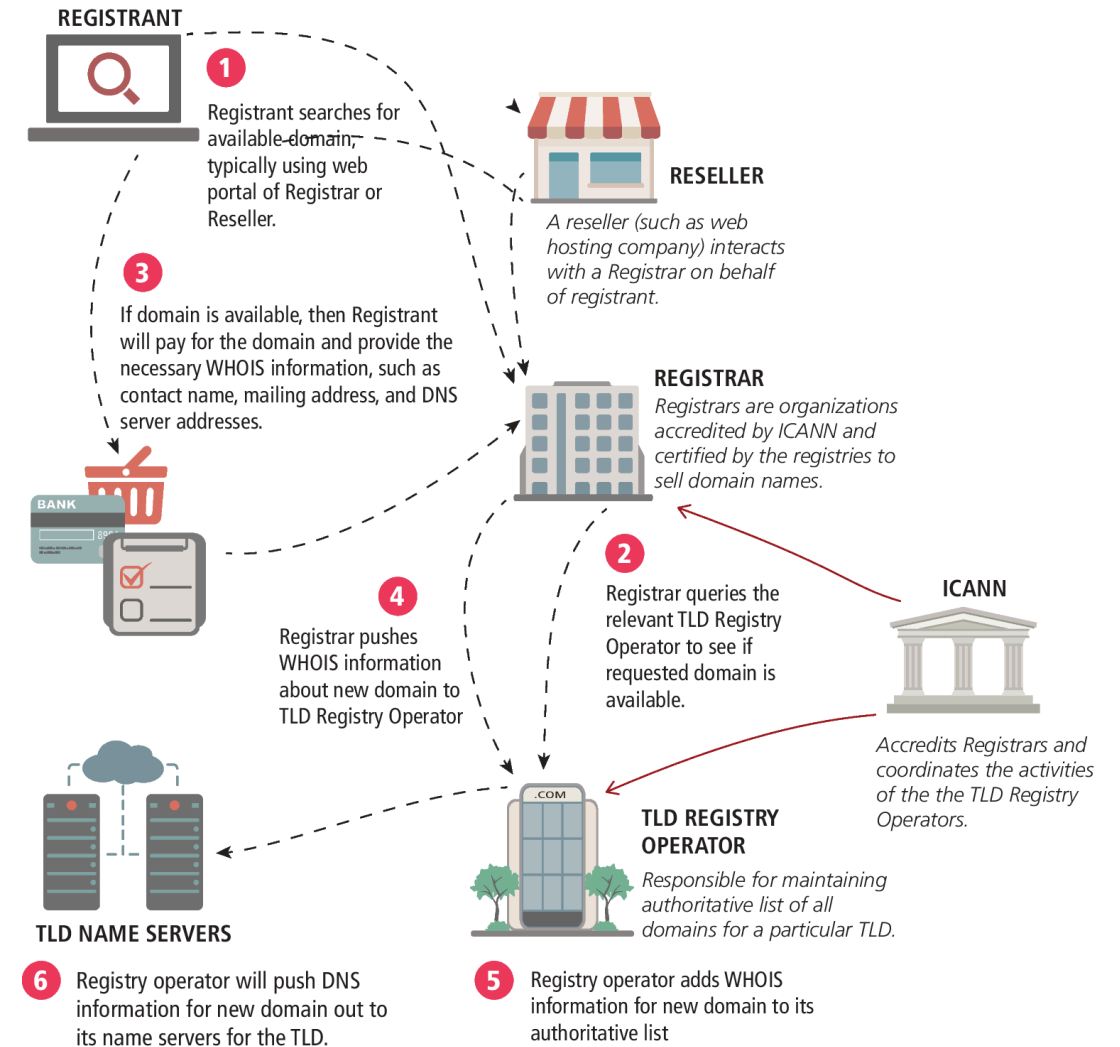
- In the United Kingdom, for example, businesses must register subdomains to **co.uk** rather than second-level domains directly whereas in Canada, **.ca** domains can be obtained by any person, company, or organization living or doing business in Canada.
- Other countries have peculiar extensions with commercial viability (such as **.tv** for Tuvalu) and have begun allowing unrestricted use to generate revenue.
- **Internationalized top-level domain name (IDN)** allows domains to use non-ascii characters and has been deployed since 2009. There are over 9 million IDN domains

Name Registration

- Q: How then are domain names assigned?
- A: Special organizations or companies called **domain name registrars** manage the registration of domain names. These domain name registrars are given permission to do so by the appropriate generic top-level domain (gTLD) registry and/or a country code top-level domain (ccTLD) registry.
- The nonprofit **Internet Corporation for Assigned Names and Numbers (ICANN)**—oversees the management of toplevel domains, accredits registrars, and coordinates other aspects of DNS.

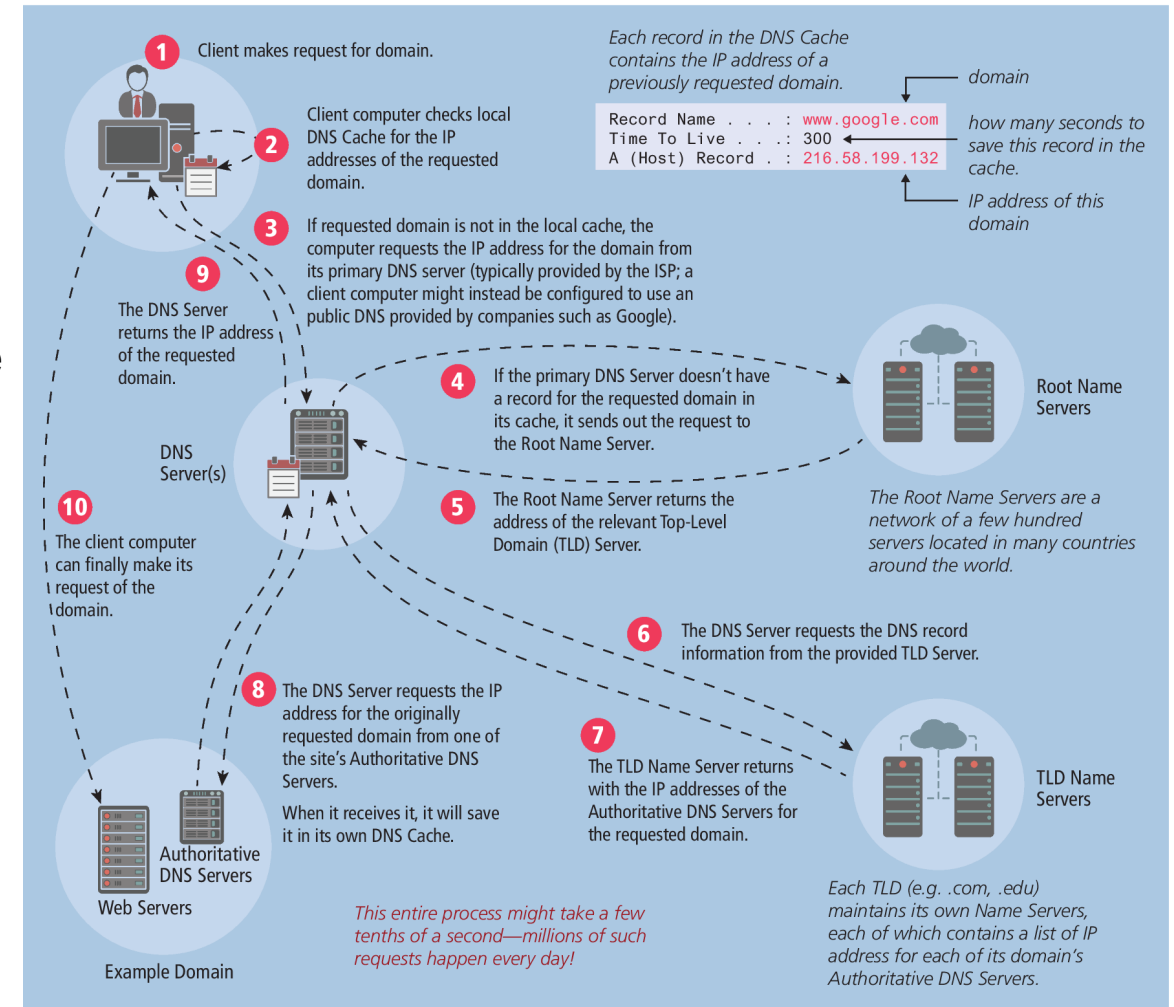
Domain name registration process

1. Registrant searches for domain, typically using web portal of Registrar or Reseller.
2. Registrar queries the relevant TLD Registry Operator to see if requested domain is available.
3. If domain is available, then Registrant will pay for the domain and provide the necessary WHOIS information
4. Registrar pushes WHOIS information about new domain to TLD Registry Operator
5. Registry operator adds WHOIS information for new domain to its authoritative list
6. Registry operator will push DNS information for new domain out to its name servers for the TLD.



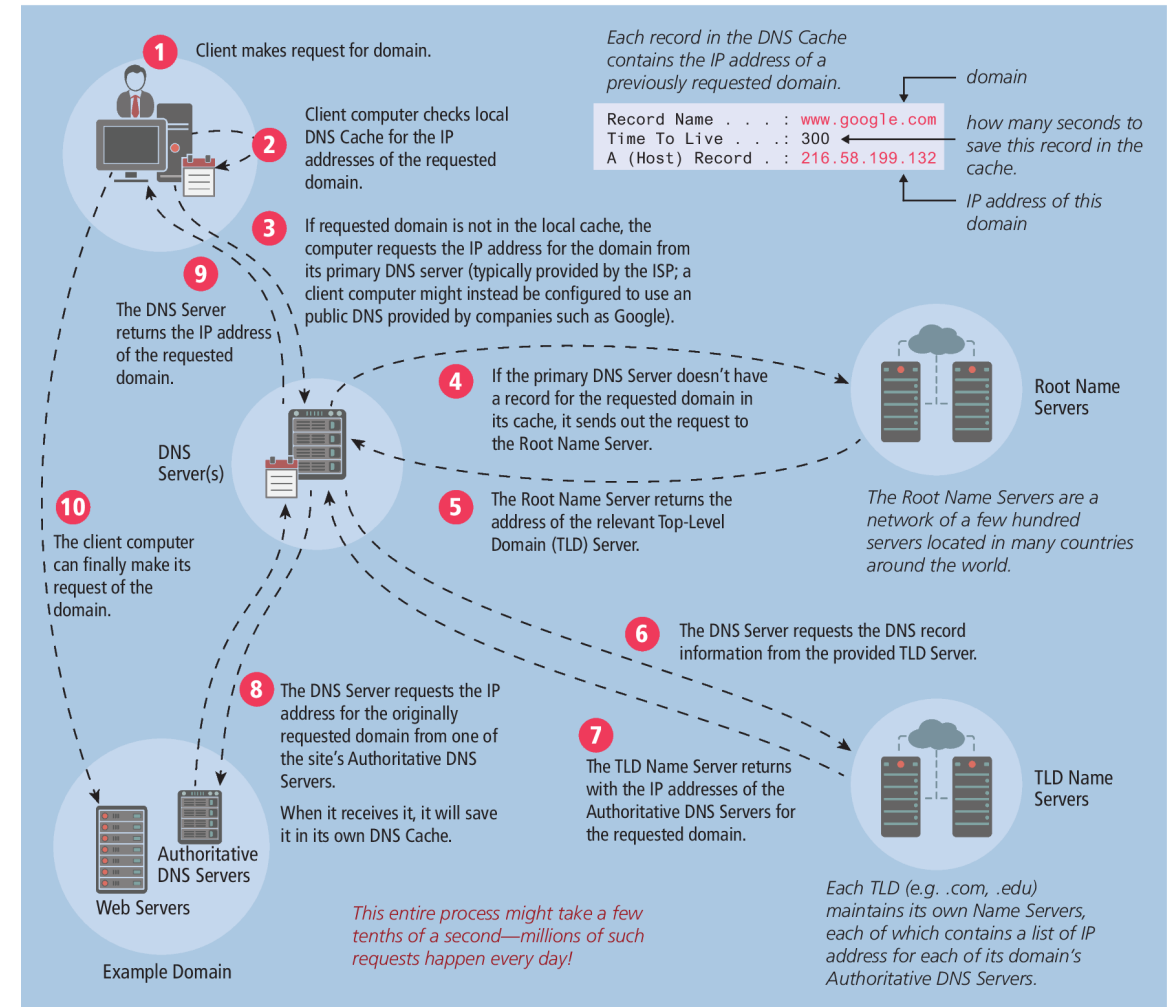
Address Resolution

1. Client makes request for domain
2. Client computer checks local DNS Cache
3. If requested domain is not in the local cache, the computer requests the IP address for the domain from its primary DNS server
4. If the primary DNS Server doesn't have a record for the requested domain in its cache, it sends out the request to the Root Name Server
5. The Root Name Server returns the address of the relevant Top-Level Domain (TLD) Server.



Address Resolution (cont)

6. The DNS Server requests the DNS record information from the provided TLD Server.
7. The TLD Name Server returns with the IP addresses of the Authoritative DNS Servers for the requested domain.
8. The DNS Server requests the IP address for the originally requested domain from one of the site's Authoritative DNS Servers.
9. The DNS Server returns the IP address of the requested domain.
10. The client computer can finally make its request of the domain.



Uniform Resource Locators

Uniform Resource Locators (URL) allow clients to request particular resources (files) from the server.

URL's consist of two required components:

1. the protocol used to connect and
2. the domain (or IP address) to connect to.

`http://www.funwebdev.com/index.php?page=17#article`

Protocol Domain Path Query String Fragment

Uniform Resource Locators (optional)

Optional components of the URL are:

- the **path** (which identifies a file or directory to access on that server),
- the **port** to connect to,
- a **query string**, and
- a **fragment** identifier

`http://www.funwebdev.com/index.php?page=17#article`

Protocol Domain Path Query String Fragment

Port (URL)

- A **port** is a type of software connection point used by the underlying TCP/IP protocol and the connecting computer.
- Although the port attribute is not commonly used in production sites, it can be used to route requests to a test server, to perform a stress test, or even to circumvent Internet filters.
- If no port is specified, the protocol determines which port to use. For instance, port 80 is the default port for web-related HTTP requests.
- Syntax is to add a colon after the domain, then specify an integer port number. <http://funwebdev.com:8080/> would connect on port 8080

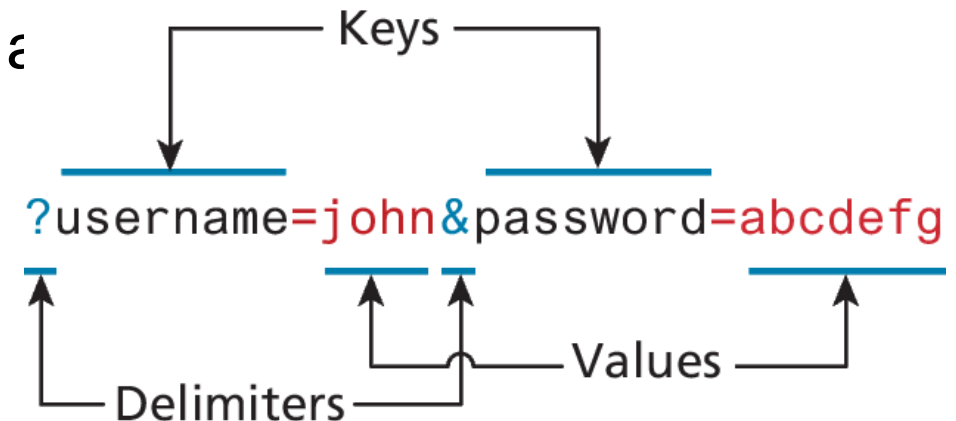
Path (URL)

- The **path** is an important concept to anyone who has ever used a computer file system.
- The root of a web server corresponds to a folder somewhere on that server. On many Linux servers that path is **/var/www/html/** or something similar (for Windows it is often **/inetpub/wwwroot/**).
- The path is optional. However, when requesting a folder or the top-level page of a domain, the web server will decide which file to send you.

Query String (URL)

Query strings will be covered in depth when we learn more about HTML forms and server-side programming. They are a critical way of passing information, such as user form input, from the client to the server.

- In URLs, they are encoded as key-value pairs delimited by & symbols and preceded by the ? Symbol
- An example query string for passing name & 2.11

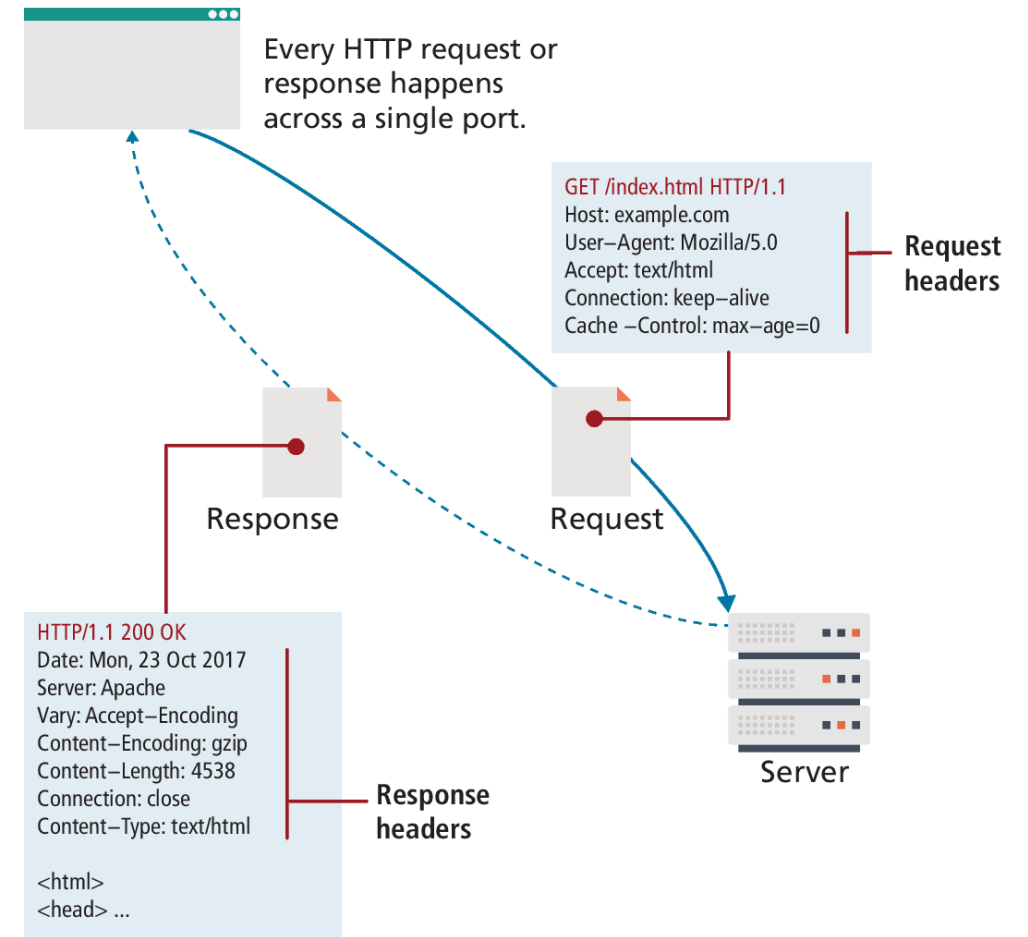


Fragment (URL)

- The last part of a URL is the optional fragment.
- This is used as a way of requesting a portion of a page.
- Browsers will see the fragment in the URL (denoted by #), seek out the fragment tag anchor in the HTML, and scroll the website down to it.
- “back to top” links are a common use of fragments.

Hypertext Transfer Protocol

- **HTTP** is an essential part of the web.
- HTTP establishes a TCP connection on port 80 (by default). The server waits for the request, and then responds with a
 - Headers,
 - Response code,
 - an optional message (which can include files)



HTTP Headers

Headers are sent in the request from the client and received in the response from the server. Headers are one of the most powerful aspects of HTTP and unfortunately, few developers spend any time learning about them.

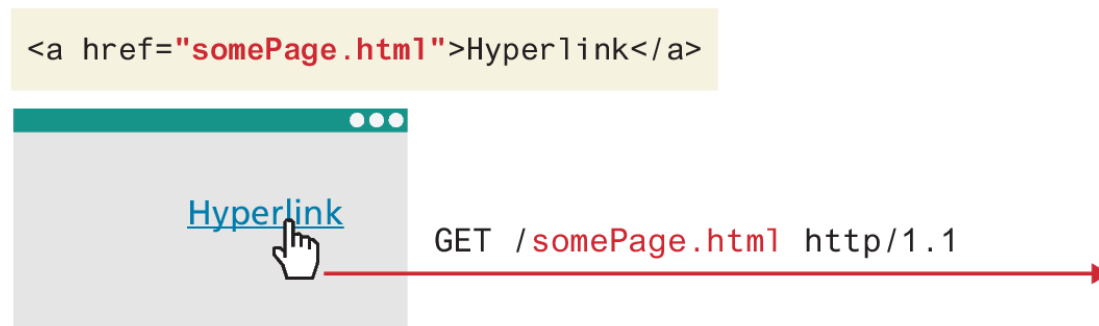
- **Request headers** include data about the client machine
 - Host, User-Agent, Cache settings and more
- **Response headers** have information about the server answering the request and the data being sent
 - Server, Last Modified, Content Type, Encoding,

HTTP Request Methods

- The most common requests are the GET and POST request, along with the HEAD request
- In Chapter 13 you will make use of the PUT and DELETE requests when creating an API in Node.
- Other HTTP verbs such as CONNECT, TRACE, and OPTIONS are less commonly used and are not covered in the book.

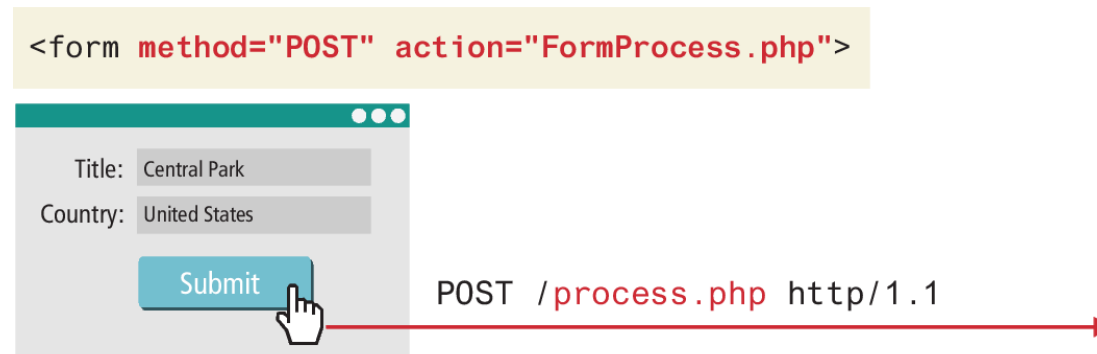
GET Request

- The most common type of HTTP request is the **GET request**.
- One is asking for a resource located at a specified URL to be retrieved.
- Whenever you click on a link, type in a URL in your browser, or click on a bookmark, you are *usually* making a GET request.
- Data can be transmitted through a GET request, with a query string



POST Request

- The other common request method is the **POST request**.
- This method is normally used to transmit data to the server using an HTML form



Response Codes

- **Response codes** are integer values returned by the server as part of the response header.
- These codes describe the state of the request, including whether it was successful, had errors, requires permission, and more.
- The codes use the first digit to indicate the category of response.
 - 2## codes are for successful responses,
 - 3## are for redirection-related responses,
 - 4## codes are client errors, while
 - 5## codes are server errors.

HTTP Response Codes (Table 2.1 edited)

Code	Description
200: OK	The request was successful.
301: Moved Permanently	Tells the client that the requested resource has permanently moved.
304: Not Modified	If the client requested a resource with appropriate Cache-Control headers, the response might say that the resource on the server is no newer than the one in the client cache.
401: Unauthorized	Some web resources are protected and require the user to provide credentials to access the resource.
404: Not found	404 codes are one of the only ones known to web users. Many browsers will display an HTML page with the 404 code to them when the requested resource was not found.
414: Request URI too long	A 414 response code likely means too much data is likely trying to be submitted via the URL.
500: Internal server error	This error provides almost no information to the client except to say the server has encountered an error.

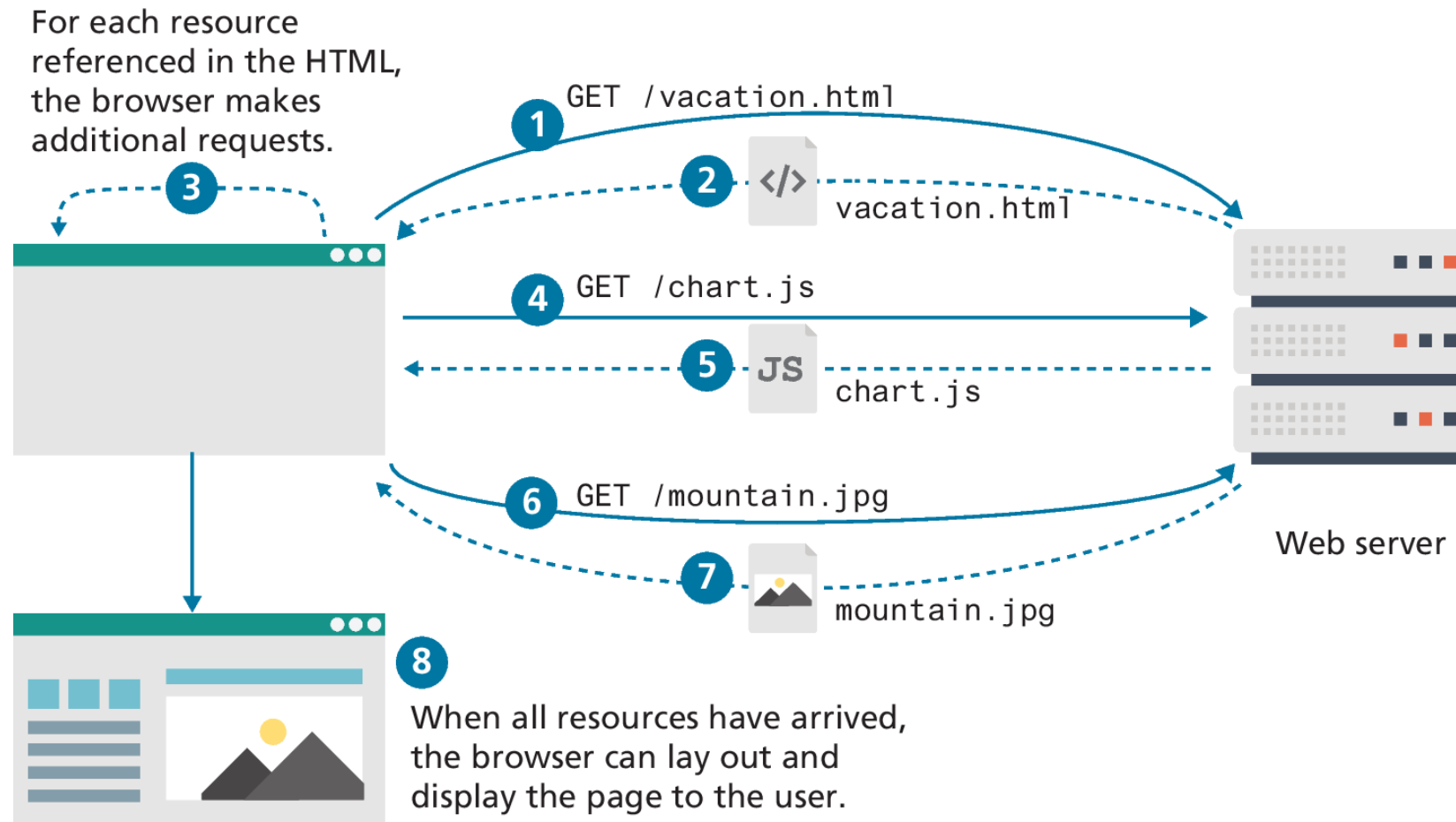
Web Browsers

- The user experience for a website is unlike the user experience for traditional desktop software.
- Users do not download software; they visit a URL, which results in a web page being displayed.
- Although a typical web developer might not build a browser, or develop a plugin, they must understand the browser's crucial role in web development.

Web Browsers (Fetching a web page)

- Seeing a single web page is facilitated by the browser, which
 - requests the initial HTML page, then
 - parses the returned HTML to find all the resources referenced from within it (like images, style sheets, and scripts).
- Only when all the files have been retrieved is the page fully loaded for the user
- A single web page can reference dozens of files and requires many HTTP requests and responses.

Fetching a web page diagram

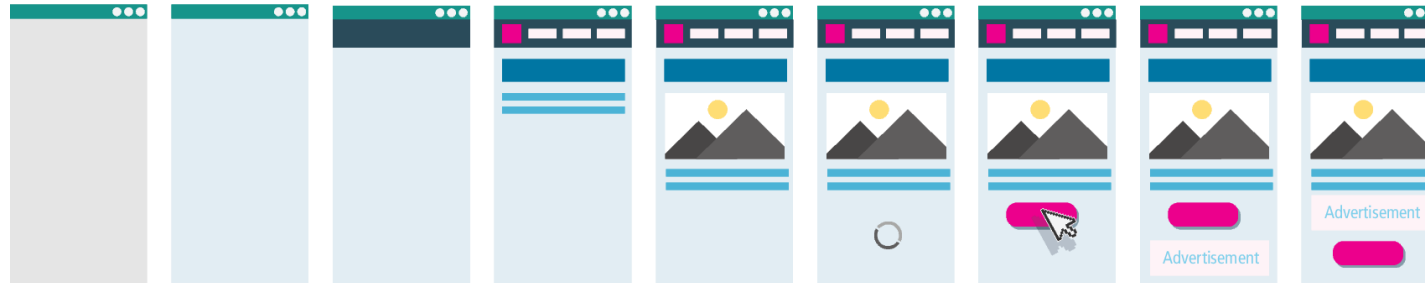


Browser Rendering

- The algorithms within browsers to download, parse, layout, fetch assets, and create the final interactive page for the user are commonly referred to collectively as the *rendering* of the page
- We will focus on the browser-rendering process through a user-centric lens where measures are categorized around perceived loading performance, interactivity, and visual stability

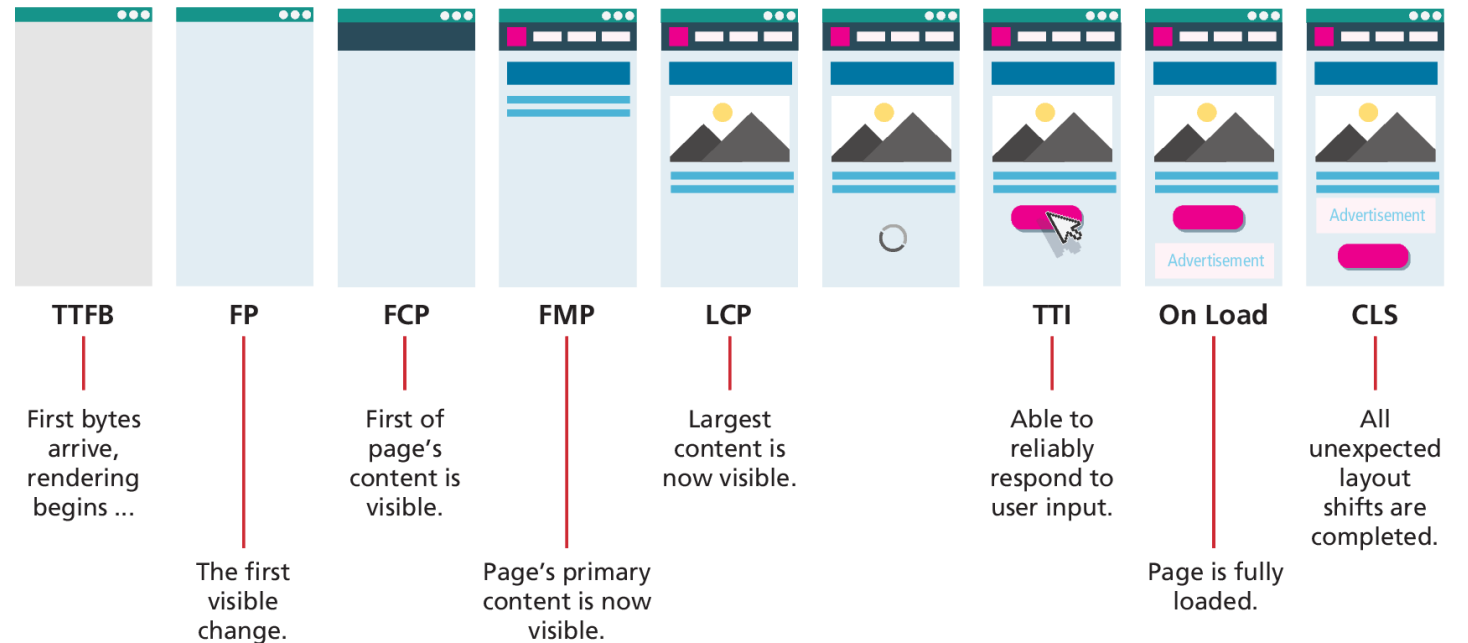
Browser Rendering (ii)

- The algorithms within browsers to download, parse, layout, fetch assets, and create the final interactive page for the user are commonly referred to collectively as the *rendering* of the page
- We will focus on the browser-rendering process through a user-centric lens where measures are categorized around perceived loading performance, interactivity, and visual stability



Browser Rendering Performance

- Time to First Byte (TTFB)
- First Paint (FP)
- First Contentful Paint (FCP)
- First Meaningful Paint (FMP)
- Largest Contentful Paint (LCP)
- Time to Interactive (TTI)
- On Load
- Cumulative Layout Shift (CLS)



Browser Caching

- Once a webpage has been downloaded from the server, it's possible that the user, a short time later, wants to see the same web page and refreshes the browser or rerequests the URL.
- Although some content might have changed, the majority of the referenced files are likely to be unchanged, so they needn't be redownloaded.
- Browser caching has a significant impact in reducing network traffic and will be come up gain in greater detail throughout this book.

Web Servers

- A **web server** is, at a fundamental level, nothing more than a computer that responds to HTTP requests.
- Real-world websites typically have many web servers configured together in web farms.
- Regardless of the physical characteristics of the server, one must choose an application stack to run a website. This **application stack** will include
 - an operating system,
 - web server software,
 - a database, and
 - a scripting language to process dynamic requests.

LAMP

- We will be using the **LAMP software stack**, which refers to the
 - **L**inux operating system,
 - **A**pache web server,
 - **M**ySQL database, and
 - **P**HP scripting language
- The Apple OSX MAMP software stack is nearly identical to LAMP, since OSX is a Unix implementation, and includes all the tools available in Linux.
- The WAMP software stack is another popular variation where Windows operating system is used.

Alternative Stacks

- Besides the LAMP stack, you will be using the **MERN stack** in the book, which refers to **M**ongoDB database, **E**xpress application framework, the JavaScript **R**ead **E**dit framework, and **N**ode.js as the web server and execution environment.
- Many corporate intranets instead make use of the Microsoft **WISA software stack**, which refers to **W**indows operating system, **I**IS web server, **S**QL Server database, and the **A**SP.NET server-side development technologies.
- Another web development stack that is growing in popularity is the so-called **JAM stack**, which refers to **J**avaScript, **A**PIs, and **m**arkup.

Key Terms

address resolution

Apache

Application stack

application layer

country code top-level

domain (ccTLD)

Cumulative Layout Shift (CLS)

DNS resolver

DNS server

domain names

domain name registrars

Domain Name System
(DNS)

First Contentful Paint (FCP)

First Meaningful Paint (FMP)

First Paint (FP)

four-layer network model

generic top-level domain
(gTLD)

GET request

google.com

HEAD request

Hypertext Transfer Protocol
(HTTP)

Internet Corporation for
Assigned Names and Numbers
(ICANN)

Internet Assigned

Numbers Authority (IANA)

internationalized top-level
domain name (IDN)

Internet layer

Internet Protocol (IP)

addresses

IP address

IPv4

IPv6

Key Terms (cont)

JAM stack

Largest Contentful Paint

LAMP software stack

link layer

MAC addresses

MEAN software stack

On Load

packet

protocol

punycode

port

Port Address Translation
(PAT)

POST request

protocol

request

request headers

response codes

response headers

reverse DNS lookups

root name server

second-level domain

subdomain

Time to First Byte (TTFB)

Time to Interactive (TTI)

transport layer

Transmission Control

Protocol (TCP)

top-level domain (TLD)

TLD name server

User Datagram Protocol (UDP)

Uniform Resource

Locator (URL)

web server

WISA software stack

Fundamentals of Web Development

Third Edition by Randy Connolly and Ricardo Hoar



Chapter 3

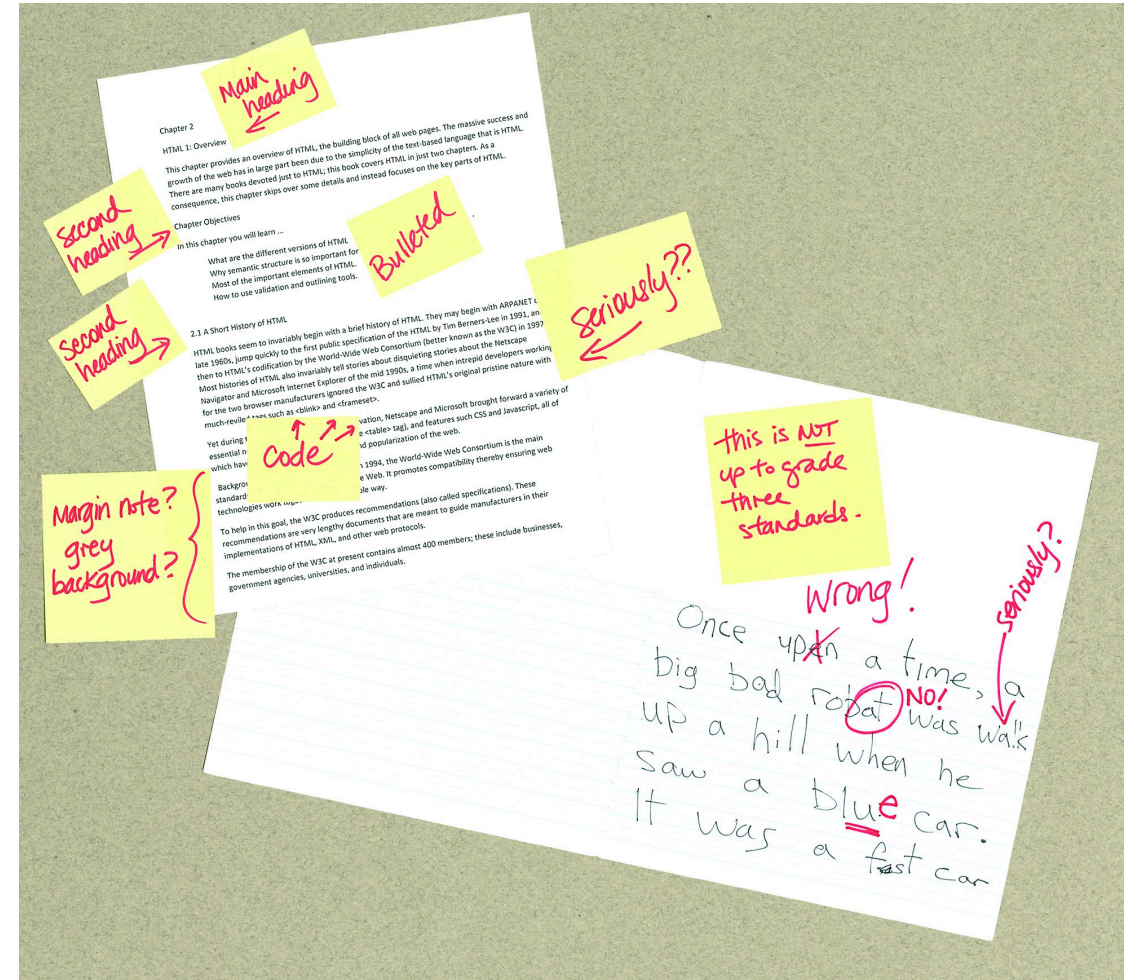
HTML 1: Introduction

In this chapter you will learn . . .

- A very brief history of HTML
- The syntax of HTML
- Why semantic structure is so important for HTML
- How HTML documents are structured
- A tour of the main elements in HTML
- The semantic structure elements in HTML5

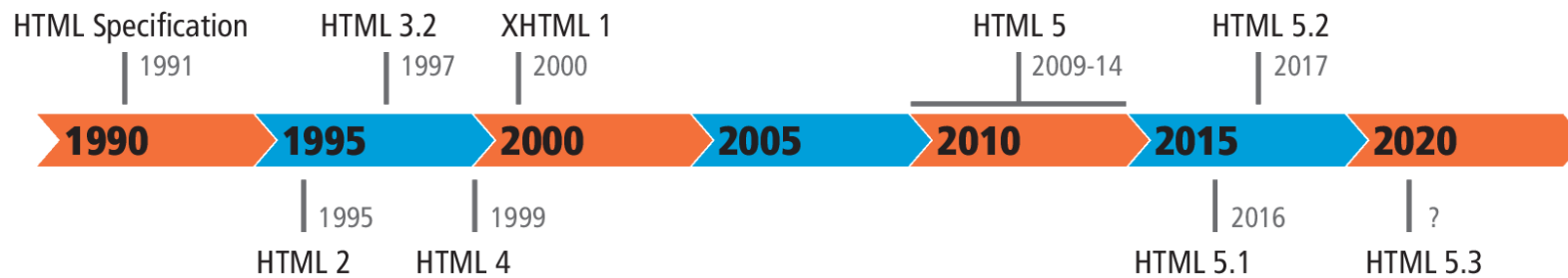
What Is HTML and Where Did It Come From?

- HTML is defined as a **markup language**. A markup language is simply a way of annotating a document in such a way as to make the annotations distinct from the text being annotated.
- You may very well have been the recipient of markup from caring parents or concerned teachers at various points in your past



HTML Markup Language

- At its simplest, **markup** is a way to indicate *information about the content* that is distinct from the content.
- This “information about content” in **HTML** is implemented via **tags** (or more formally, HTML elements, but more on that later).
- Since the initial HTML specification in 1991, HTML has gone through many interesting changes worth understanding in brief.



Early HTML

- Initial implementation of HTML and HTTP between 1990 and 1991 by Tim Berners-Lee and Robert Cailliau
- HTML's formal codification by the **World Wide Web Consortium** (better known as the **W3C**) between 1995 and 1997
- “browser wars” in the mid 1990s between Netscape Navigator and Microsoft Internet Explorer motivated many new tags and features such as CSS and JavaScript, but the development of new features happened quickly, and interoperability between browsers became a major issue
- In 1998 the W3C froze the HTML specification at version 4.01

XHTML

- In the late 1990s the W3C developed o a new specification called **XHTML 1.0**, which was a version of HTML that used stricter **XML**
- The goal of XHTML with its strict rules was to make page rendering more predictable by forcing web authors to create web pages without syntax errors.
- To help web authors, two versions of XHTML were created:
 - XHTML 1.0 Strict and
 - XHTML 1.0 Transitional.
- Development on the XHTML 2.0 specification dragged on for many years

HTML5

- While the XHTML 2.0 specification was being developed, a small group of developers at Opera and Mozilla formed the **WHATWG** (Web Hypertext Application Technology Working Group).
- By 2009, the W3C stopped work on XHTML 2.0 and instead adopted the work done by WHATWG and named it **HTML5**.
- There are three main aims to HTML5:
 1. Specify unambiguously how browsers should deal with invalid markup.
 2. Provide an open, nonproprietary programming framework (via JavaScript) for creating rich web applications.
 3. Be backward compatible with the existing web.

HTML Syntax

- **HTML documents** are composed of **content** and HTML elements.
- An **HTML element** is identified in the HTML document by tags.
 - A **tag** consists of the element name within angle brackets.
- The **element name** appears in both the **opening tag** and the **closing tag**.
- HTML elements can also contain attributes. An **HTML attribute** is a name=value pair that provides more information about the element



Empty Element

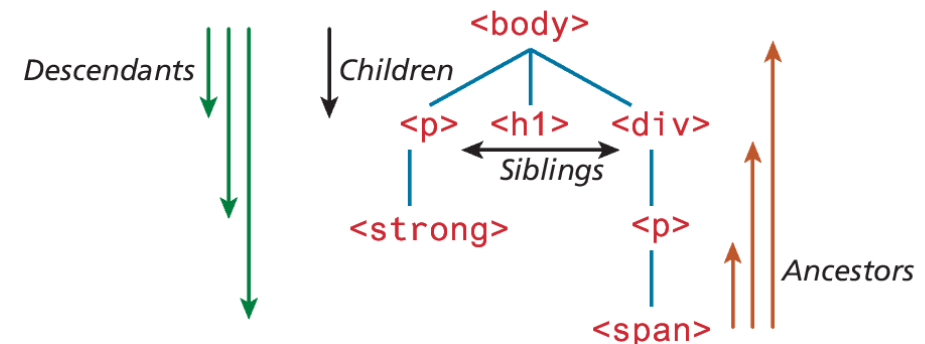
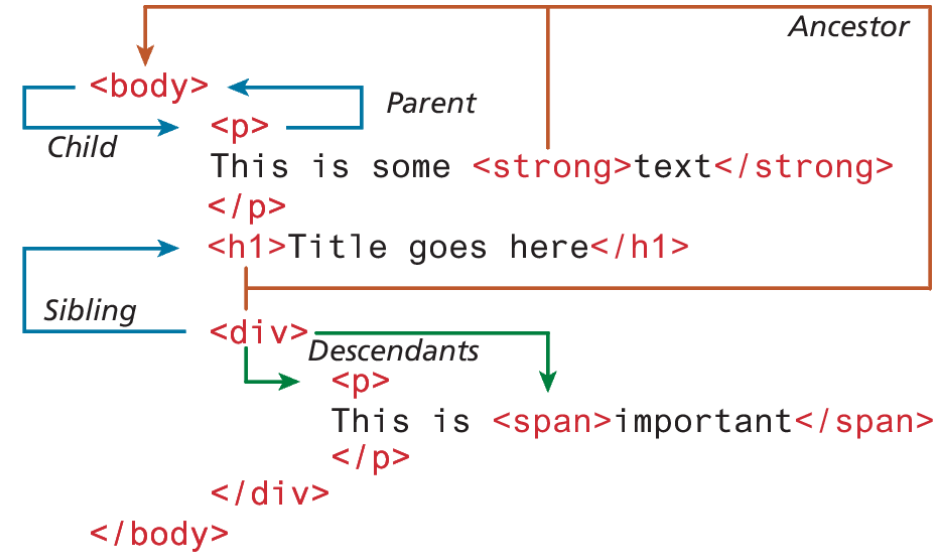
- An **empty element** does not contain any text content; instead, it is an instruction to the browser to do something.
- Perhaps the most common empty element is `<img>`, the image element.
- In XHTML, empty elements had to be terminated by a trailing slash (as shown in image). In HTML5, the trailing slash in empty elements is optional.

Example empty element ``

`<` Element name `/>` Trailing slash (*optional*)

Nesting HTML Elements

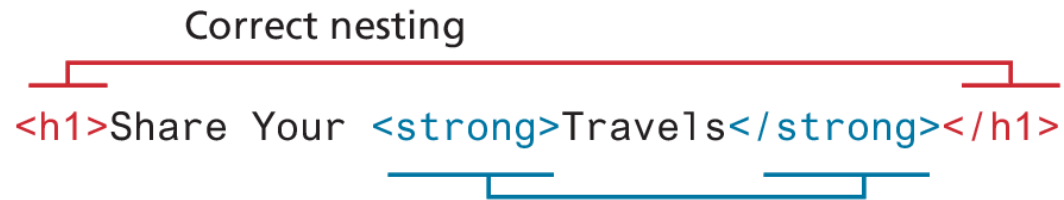
- Often an HTML element will contain other HTML elements. In such a case, the container element is said to be a **parent** of the contained, or **child**, element.
- Any elements contained within the child are said to be **descendants** of the parent element; likewise, any given child element may have a variety of **ancestors**.
- This concept is called the **Document Object Model** (DOM) formally, though for now we will only refer to its hierarchical aspects.



Correct Nesting

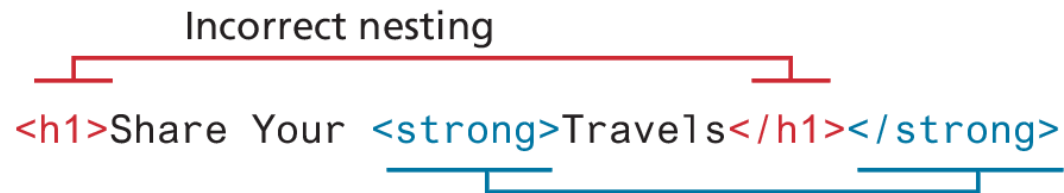
- In order to properly construct this hierarchy of elements, your browser expects each HTML nested element to be properly nested.
- A child's ending tag must occur before its parent's ending tag

Correct nesting



```
<h1>Share Your <strong>Travels</strong></h1>
```

Incorrect nesting



```
<h1>Share Your <strong>Travels</h1></strong>
```

Semantic Markup

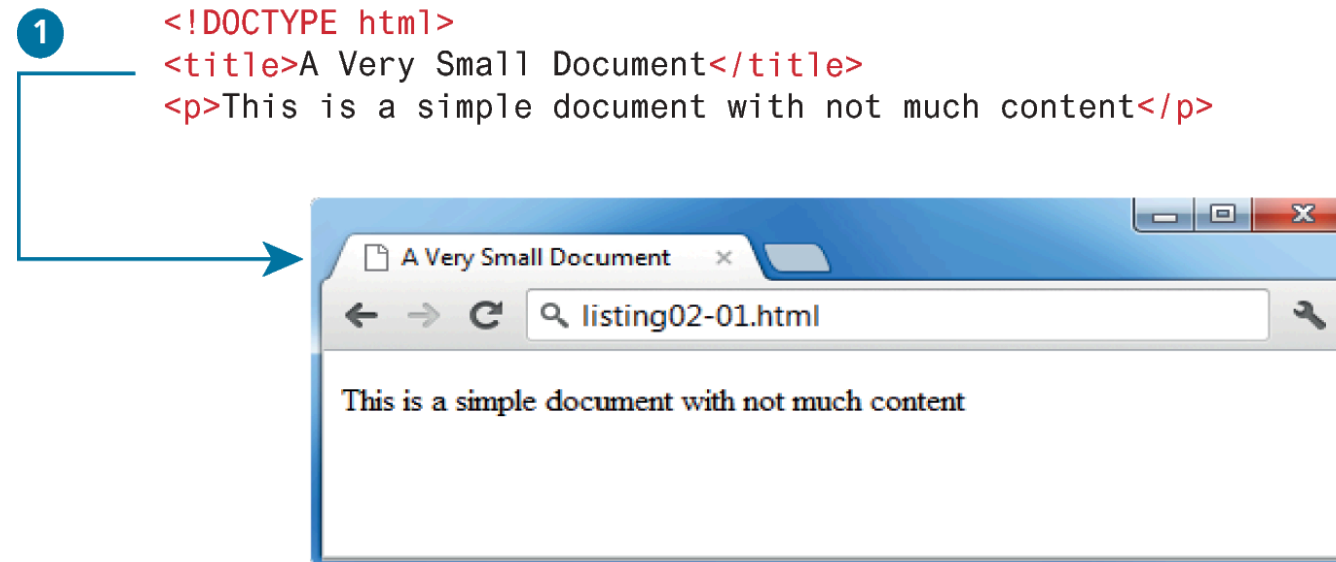
- HTML documents should **only** focus on the structure of the document
- Information about how the content should look is best left to CSS (Cascading Style Sheets), a topic introduced in the next chapter
- As a consequence, beginning HTML authors are often counseled to create **semantic HTML** documents.
- HTML document should not describe how to visually present content but only describe its content's structural semantics or meaning

Semantic Markup Advantages

- **Maintainability**. Semantic markup is easier to update and change than web pages that contain a great deal of presentation markup.
- **Performance**. Semantic web pages are typically quicker to author and faster to download.
- **Accessibility**. Not all web users are able to view the content on web pages. Users with sight disabilities experience the web using voice-reading software.
- **Search engine optimization**. For many site owners, the most important users of a website are the various search engine crawlers.

Structure of HTML Documents

- Figure 3.8 illustrates one of the simplest *valid* HTML5 documents you can create.



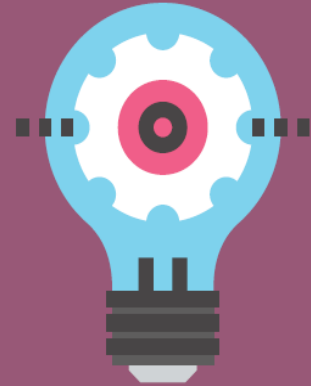
The title element

PRO TIP

The **<title>** element plays an important role in search engine optimization (SEO), that is, improving a page's rank (its position in the results page after a search) .

While each search engine uses different algorithms for determining a page's rank, the **title** (and the major headings) provides a key role in determining what a given page is about.

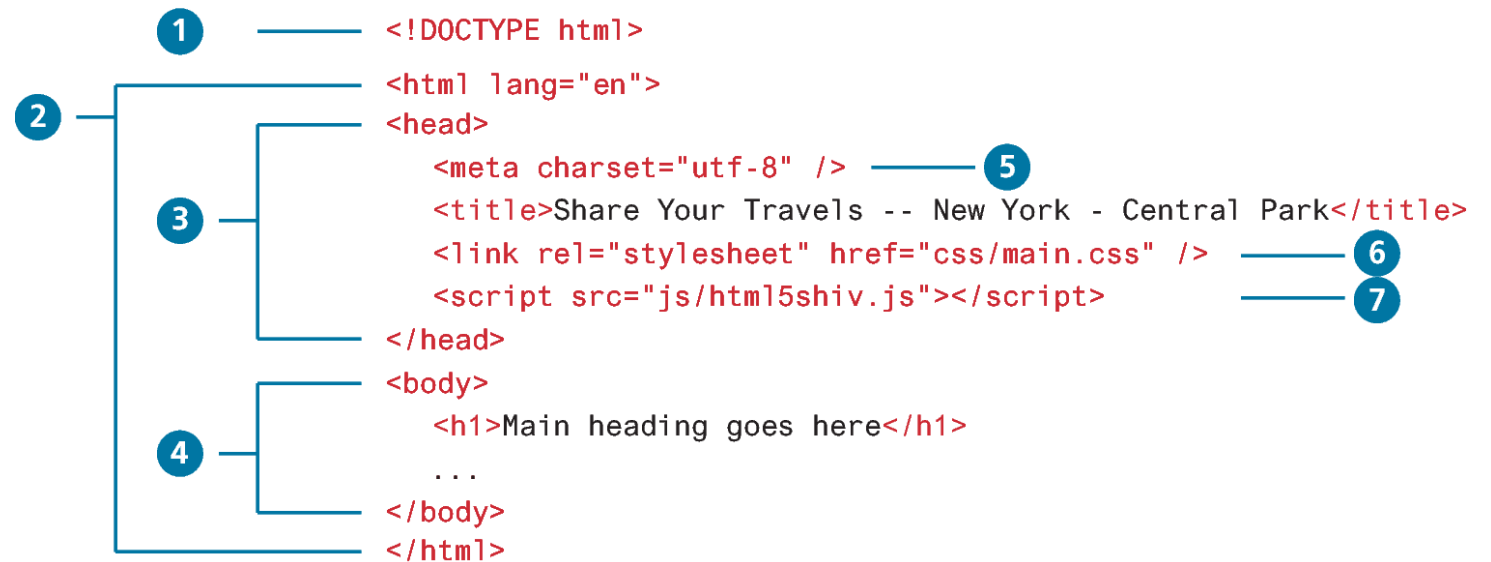
As a result, be sure that a page's title text briefly summarizes the document's content. As well, put the most important content first in the title. Most browsers limit the length of the title that is displayed in the tab or window title to about 60 characters. Chapter 18 goes into far greater detail on SEO.



Structure elements of an HTML5 document

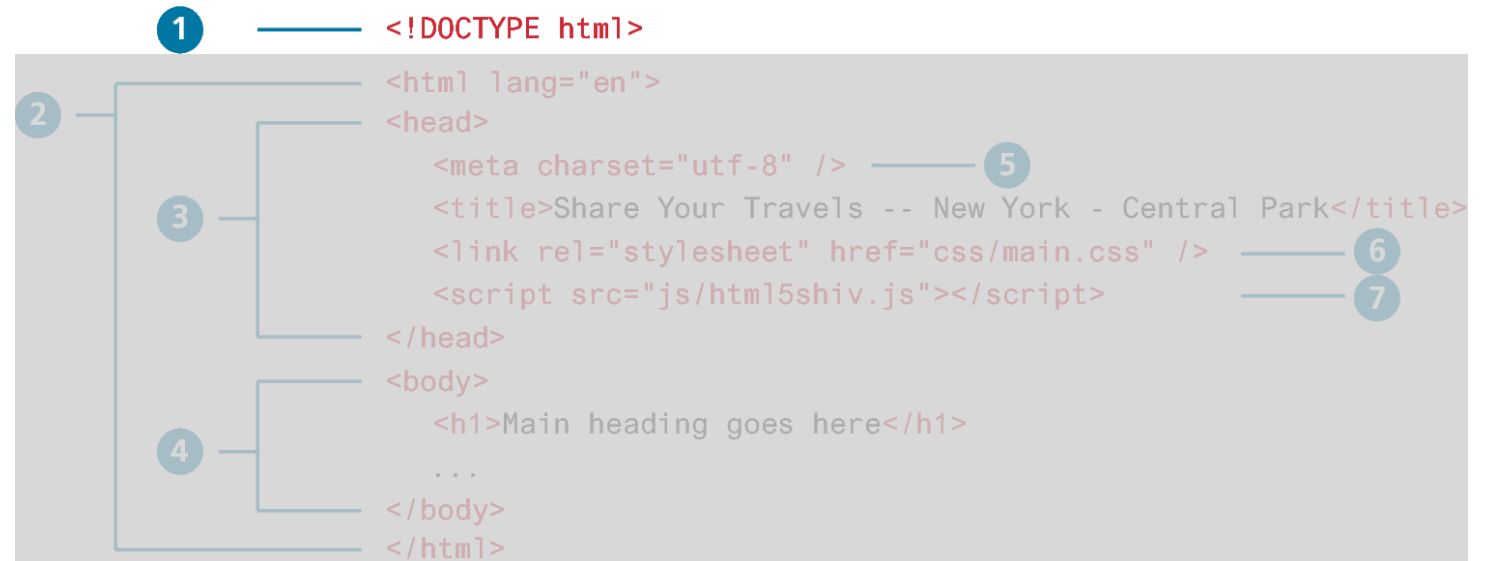
- Consider this more complete HTML5 document that includes structural elements as well as some other common HTML elements.

- Let's explore this page in detail



DOCTYPE

- The DOCTYPE declaration tells the browser what type of document it is about to process.
- It does not indicate what version of HTML is contained within the document



<Html> element

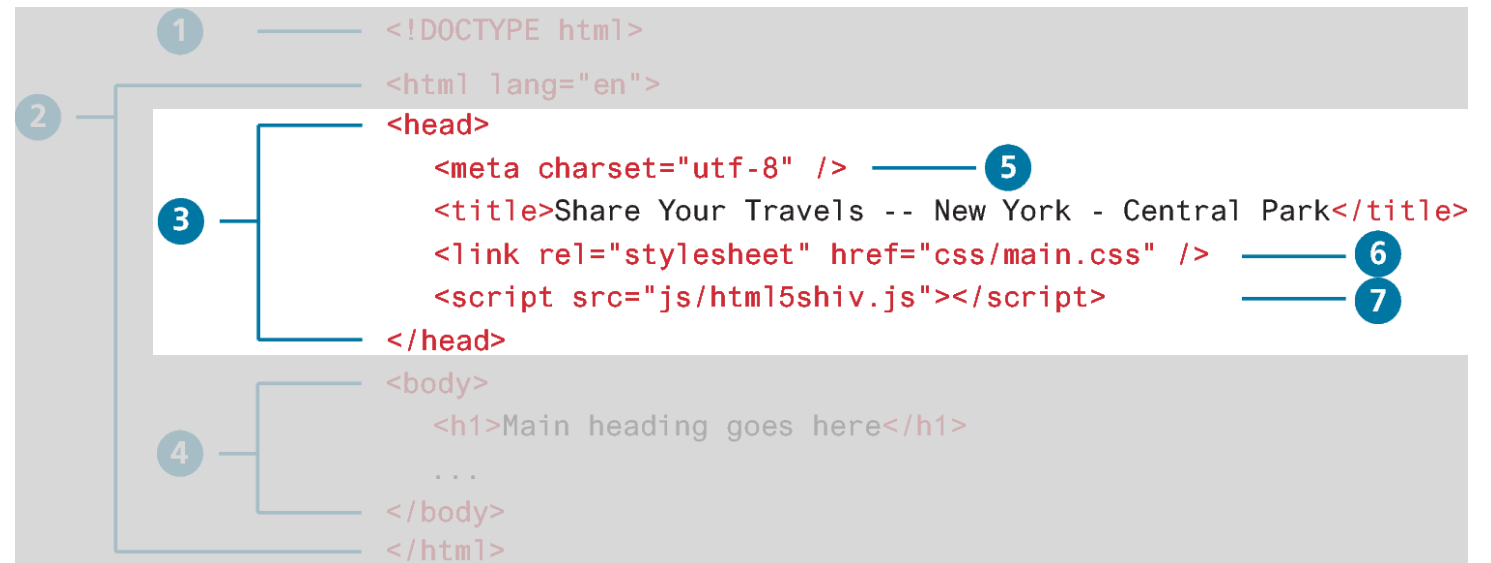
HTML5 does not require the use of the <html>, <head>, and <body> elements. However, in XHTML they were required, and most web authors continue to use them.

The <html> element is sometimes called the **root element** as it contains all the other HTML elements in the document. The optional **lang** attribute tells the browser the language that is being used.



<Head> Element

The **head** contains descriptive elements *about* the document, such as its title, encoding, and any style sheets or JavaScript files it uses.



<Body> Element

The body contains content (both HTML elements and regular text) that will be displayed by the browser. The rest of this chapter and the next chapter will cover the HTML that will appear within the body.



Quick Tour of HTML Elements

1. **Headings.** Describes the main structure of document. There are six levels of headings.
2. **Paragraphs.** The basic unit of text in HTML. As block-level elements, browsers typically add newlines before and after the element.
3. **Link.** Hyperlinks are essential feature of all web pages and can reference another page or another location in same page.
4. **Inline Text Elements.** These do not change the flow of text and provide more information about text.
5. **Image.** Used to display an image by specifying a filename or URL.

```

<body>
1  <h1>Share Your Travels</h1>
2  <h2>Venice - Grand Canal</h2>
3  <p>Photo by Randy Connolly</p>
4  <p>This view of the Grand Canal in
   <a href="https://en.wikipedia.org/wiki/Venice">Venice</a>
   was taken from the <strong>Ponte di Rialto</strong>.
5  </p>
6  
7  <ul>
   <li>Photo by <em>Randy Connolly</em></li>
   <li>Take on June 23, 2017</li>
</ul>
8  <h3>Reviews</h3>
9  <div>
   <p>By Hypatia on <time>2019-10-23</time></p>
   <p>I love Venice in the morning.</p>
</div>
10 <hr>
    <div>
       <p>By Curia on <time>2019-12-11</time></p>
       <p>I want to visit Venice!</p>
    </div>
    <footer>Copyright &copy; 2020 Share Your Travels</footer>
</body>

```

Quick Tour of HTML Elements (cont)

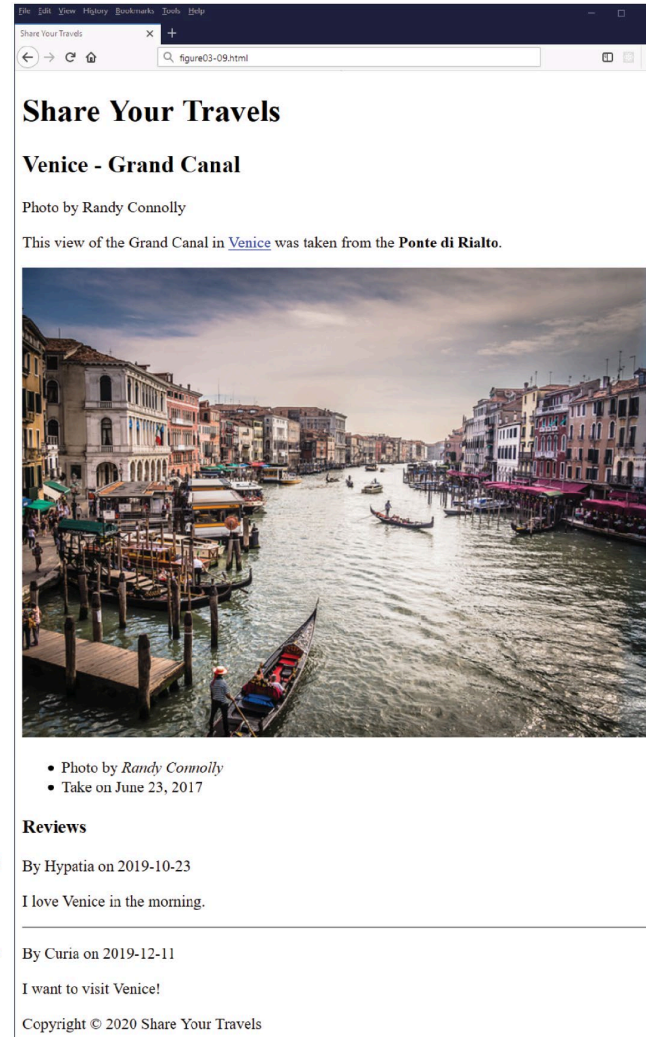
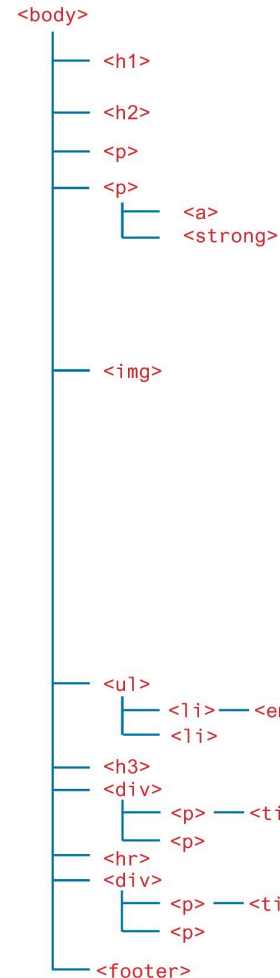
6. **Unordered List.** Used to display a bulleted list. Within a list is a collection of list item elements.
7. **Division.** Container for text or other HTML elements. Like paragraphs, they are also block-level elements.
8. **Horizontal Rule.** Indicates a thematic break in the text. Usually displayed as a horizontal line.
9. **Character Entity.** The mechanism for including special symbols (such as ©) or characters that have a reserved meaning in HTML.
10. **Semantic Block Element.** Special containers in HTML5 for describing structural elements in a document.

```

<body>
1  <h1>Share Your Travels</h1>
2  <h2>Venice - Grand Canal</h2>
   <p>Photo by Randy Connolly</p>
   <p>This view of the Grand Canal in
     <a href="https://en.wikipedia.org/wiki/Venice">Venice</a> 3
     was taken from the <strong>Ponte di Rialto</strong>.
   </p>
5  
6  <ul>
   <li>Photo by <em>Randy Connolly</em></li>
   <li>Take on June 23, 2017</li>
</ul>
   <h3>Reviews</h3>
7  <div>
   <p>By Hypatia on <time>2019-10-23</time></p>
   <p>I love Venice in the morning.</p>
</div>
   <hr> 8
   <div>
     <p>By Curia on <time>2019-12-11</time></p>
     <p>I want to visit Venice!</p>
   </div> 9
   <footer>Copyright &copy; 2020 Share Your Travels</footer>
</body> 10

```

In the browser

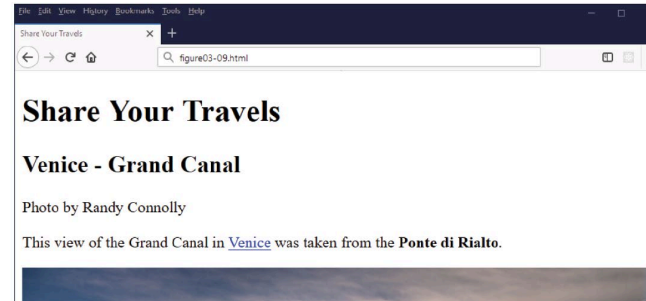


In the browser (note)

```

<body>
├── <h1>
├── <h2>
├── <p>
├── <p>
│   ├── <a>
│   └── <strong>

```



NOTE

Why does this look so awful? Plain HTML is just that . . . plain looking.

To make our pages look more stylish, you need to style the elements using CSS, which you will learn in Chapters 4 and 7.



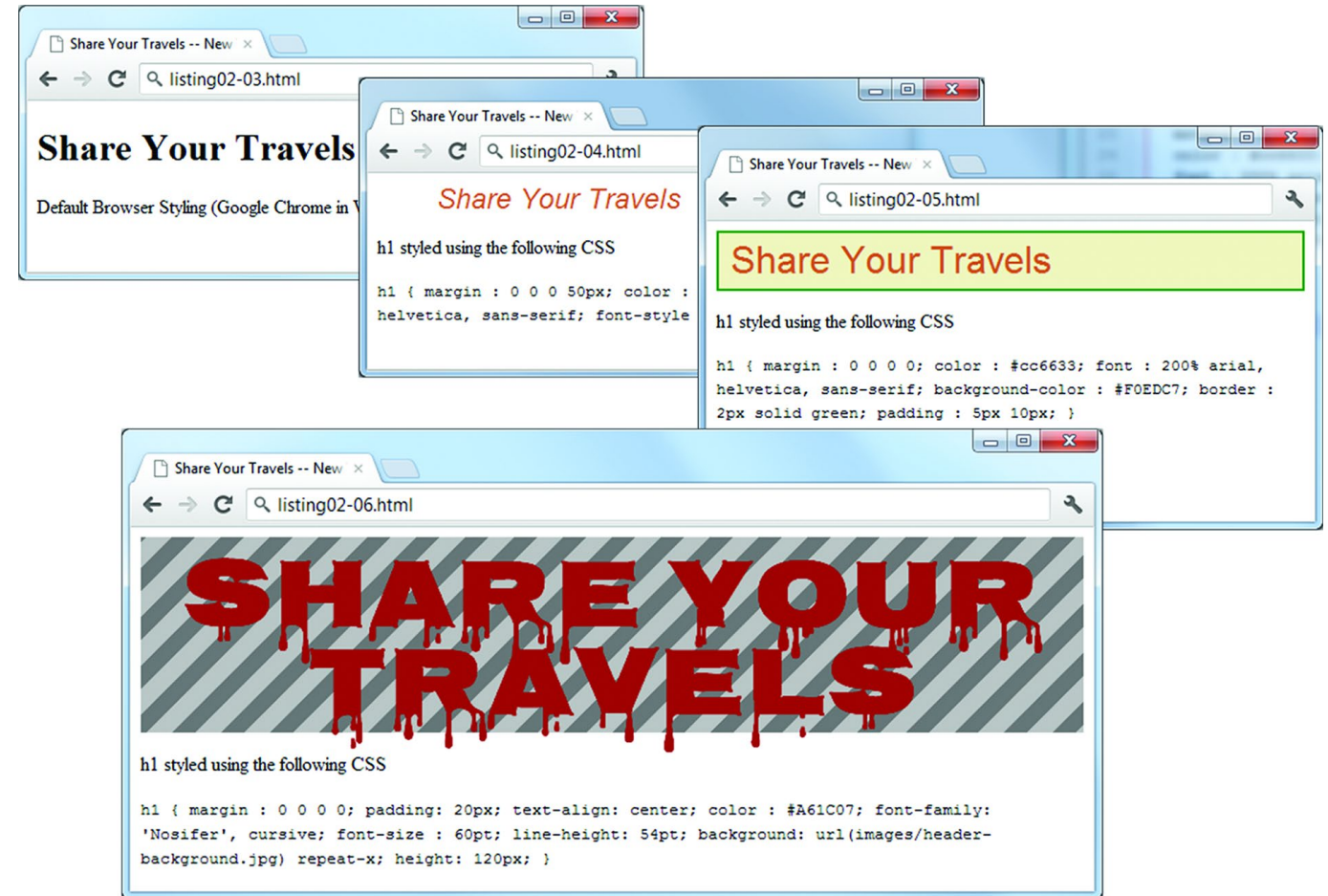
Headings

- HTML provides six levels of heading (h1 through h6)
- They are an essential way for document authors to show their readers the structure of the document
- Headings are also used by the browser to create a **document outline** for the page.
- Choose the heading level because it is appropriate semantically NOT because of its default presentation (e.g., choosing <h3> because you want your text to be bold and 16pt).

Heading Styles

The browser has its own default styling for each heading level.

These are easily modified and customized via CSS (next chapter)



Paragraphs and Divisions

- The `<p>` tag is a container. It can contain HTML and other **inline HTML elements**
- `<div>` is also a container element The `<div>` element has no intrinsic presentation or semantic value;
- `<hr>` element is used to add a “break” between paragraphs or `<div>` elements.

```

<body>
  1 — | <h1>Share Your Travels</h1>
      | <h2>Venice - Grand Canal</h2>
  2 — | <p>Photo by Randy Connolly</p>
      | <p>This view of the Grand Canal in
      |   <a href="https://en.wikipedia.org/wiki/Venice">Venice</a> 3
      |   was taken from the <strong>Ponte di Rialto</strong>.
      | </p>
  5 — | 
  6 — | <ul>
      |   <li>Photo by <em>Randy Connolly</em></li>
      |   <li>Take on June 23, 2017</li>
      | </ul>
      | <h3>Reviews</h3>
  7 — | <div>
      |   <p>By Hypatia on <time>2019-10-23</time></p>
      |   <p>I love Venice in the morning.</p>
      | </div>
      | <hr> 8
      | <div>
      |   <p>By Curia on <time>2019-12-11</time></p>
      |   <p>I want to visit Venice!</p>
      | </div> 9
  10 — | <footer>Copyright &copy; 2020 Share Your Travels</footer>
      | </body>

```

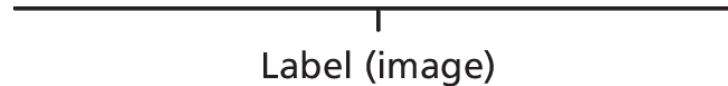

HyperLinks

- Links are created using the `<a>` element (the “a” stands for anchor).
- A link has two main parts: the **destination** and the **label**.

```
<a href="http://www.centralpark.com">Central Park</a>
```



```
<a href="index.html"></a>
```



Kinds of Links

- Links to external sites (or to individual resources, such as images or movies on an external site).
- Links to other pages or resources within the current site.
- Links to other places within the current page.
- Links to particular locations on another page (whether on the same site or on an external site).
- Links that are instructions to the browser to start the user's email program.
- Links that are instructions to the browser to execute a JavaScript function.
- Links that are instructions to the mobile browser to make a phone call.
- Links that are instructions to other programs (e.g., Skype, FaceTime, FaceBook Messenger).

Absolute and Relative URLs

When referencing a page or resource on an **external site**, a full **absolute URL reference** is required

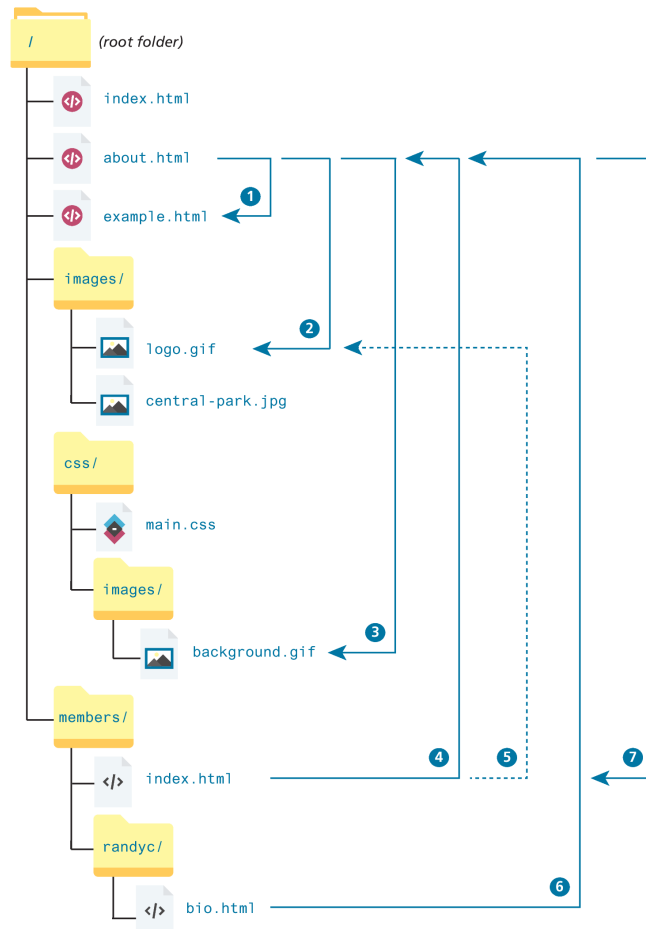
- Full URL with a protocol (typically, http:// or https://), the domain name, any paths, and the file name of the desired resource.

When referencing a resource that is on the **same server**, you can use **relative referencing**.

- If the URL does not include the “**http://**” then the browser will request the current server for the file.

Relative URLs

Share-Your-Travels



1. **Same Directory** To link to a file within the same folder, simply use the file name.
2. **Child Directory** To link to a file within a subdirectory, use the name of the subdirectory and a slash before the file name.
3. **Grandchild/Descendant Directory** To link to a file that is multiple subdirectories *below* the current one, construct the full path by including each subdirectory name (separated by slashes) before the file name.
4. **Parent/Ancessor Directory** Use “../” to reference a folder *above* the current one. If trying to reference a file several levels above the current one, simply string together multiple “../”.
5. **Sibling Directory** Use “../” to move up to the appropriate level, and then use the same technique as for child or grandchild directories.
6. **Root Reference** In this approach, begin the reference with the root reference (the “/”), and then use the same technique as for child or grandchild directories.

See Table 3.1 for examples (p. 97)

Inline Text Elements

- inline elements because they do not disrupt the flow of text (i.e., cause a line break).
- HTML defines over 30 of these elements.
- Table 3.2 lists some of the most commonly used of these elements.

Common Text-Level Semantic Elements

- **<a>** Anchor used for hyperlinks.
- **<abbr>** An abbreviation
- **
** Line break
- **<cite>** Citation (i.e., a reference to another work)
- **<code>** Used for displaying code, such as markup or programming code
- **** Emphasis
- **<mark>** For displaying highlighted text
- **<small>** For displaying the fine-print, that is, “nonvital” text, such as copyright or legal notices
- **** The inline equivalent of the <div> element. It is generally used to mark text that will receive special formatting using CSS
- **** For content that is strongly important
- **<time>** For displaying time and date data

Images

- Chapter 6 examines the different types of graphic file formats.
- Note the key attributes of the `<img>` element.
- Attributes such as `title`, `width`, and `height` are optional

Specifies the URL of the image to display
(note: uses standard relative referencing).

Text in `title` attribute will be displayed in a pop-up
tool tip when user moves mouse over image (optional).

```

```

Text in `alt` attribute provides a brief
description of image's content for users who
are unable to see it.

Specifies the width and height of
image in pixels (discouraged)

Character Entities

Character entities are special characters for symbols for which there is either no easy way to type them via a keyboard or which have a reserved meaning in HTML (for instance the “<” or “>” symbols).

- They can be used in an HTML document by using the entity name or the entity number

Entity Examples

Entity Name	Entity Number	Description
&nbsp;	&#160;	Nonbreakable space.
&lt;	&#60;	Less than symbol (“<”).
&gt;	&#62;	Greater than symbol (“>”).
&copy;	&#169;	The © copyright symbol
&euro;	&#8364;	The € euro symbol.
&trade;	&#8482;	The ™ trademark symbol.

Lists

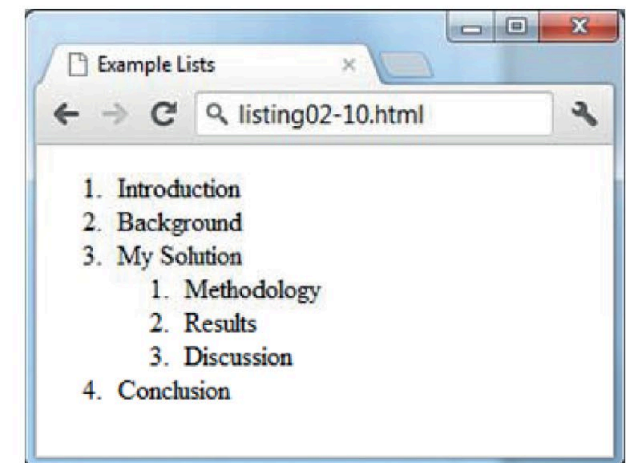
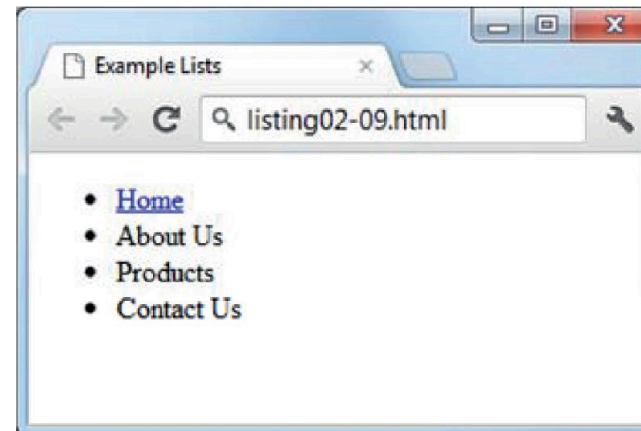
- **Ordered lists** Collections of items that have a set order
`<ol>` `<li>`
- **Unordered Lists**
 Collections of items in no particular order
`<ul>` `<li>`
- **Description Lists**
 Collection of name and description/definition pairs.
`<dl>` `<dt>`

Notice that the list item element can contain other HTML elements.

```

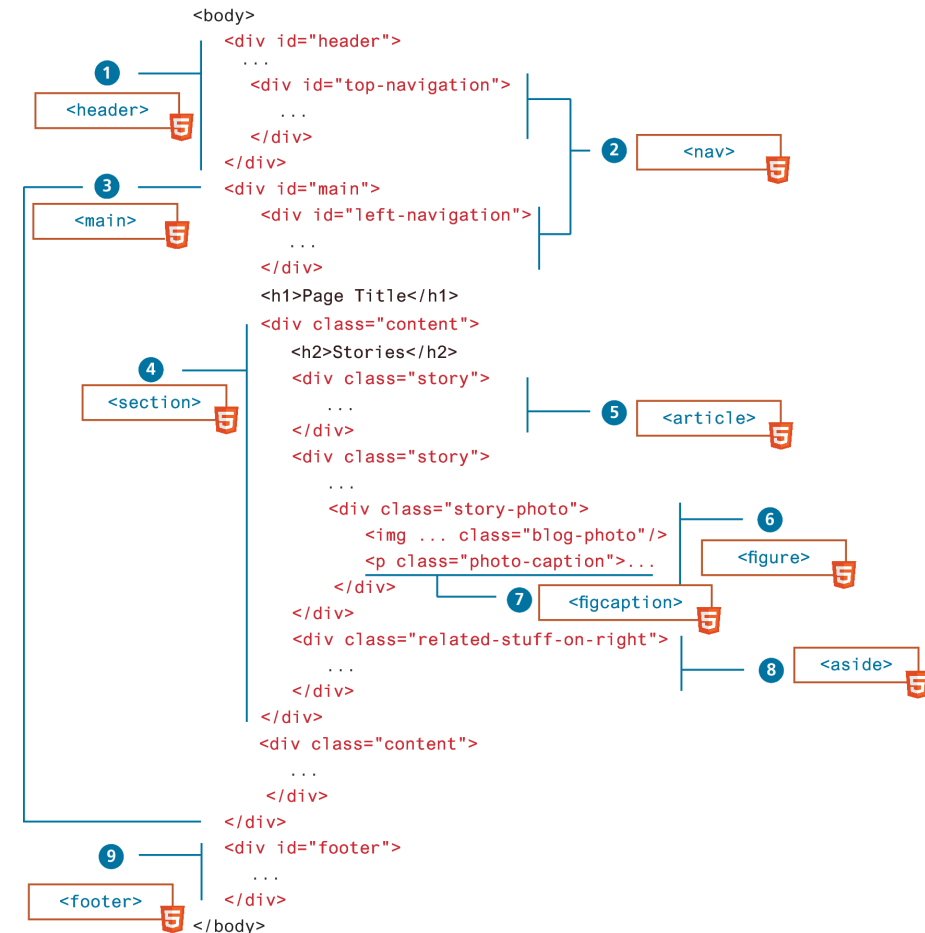
<ul>
  <li><a href="index.html">Home</a></li>
  <li>About Us</li>
  <li>Products</li>
  <li>Contact Us</li>
</ul>

<ol>
  <li>Introduction</li>
  <li>Background</li>
  <li>My Solution</li>
  <li>
    <ol>
      <li>Methodology</li>
      <li>Results</li>
      <li>Discussion</li>
    </ol>
  </li>
  <li>Conclusion</li>
</ol>
  
```



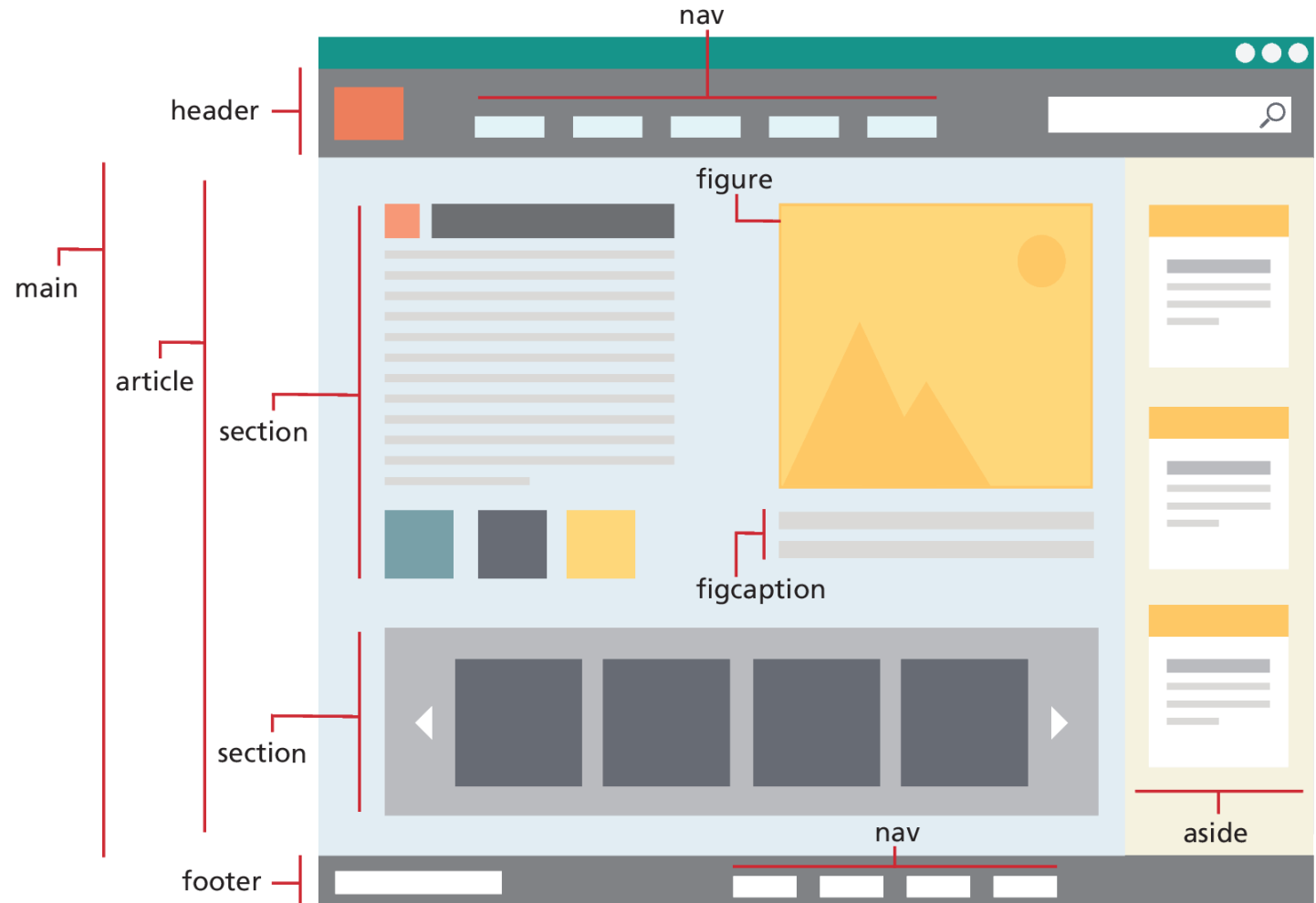
HTML5 Semantic Structure

- So far, the main semantic elements you have seen are headings, paragraphs, lists, some inline elements and the semantic block element, the division (i.e., `<div>` element).
- HTML5 semantic elements allow to replace some of your `<div>` sprawl with cleaner and more self-explanatory elements



HTML5 Semantic Structure Elements

- Header
- Nav
- Main
- Section
- Article
- Figure
- Figcaption
- Aside
- Footer



Using Semantic Elements

- HTML5 semantic elements do not apply any special presentation giving them great flexibility.
- Article and section, for instance, can be used many ways when designing your website.

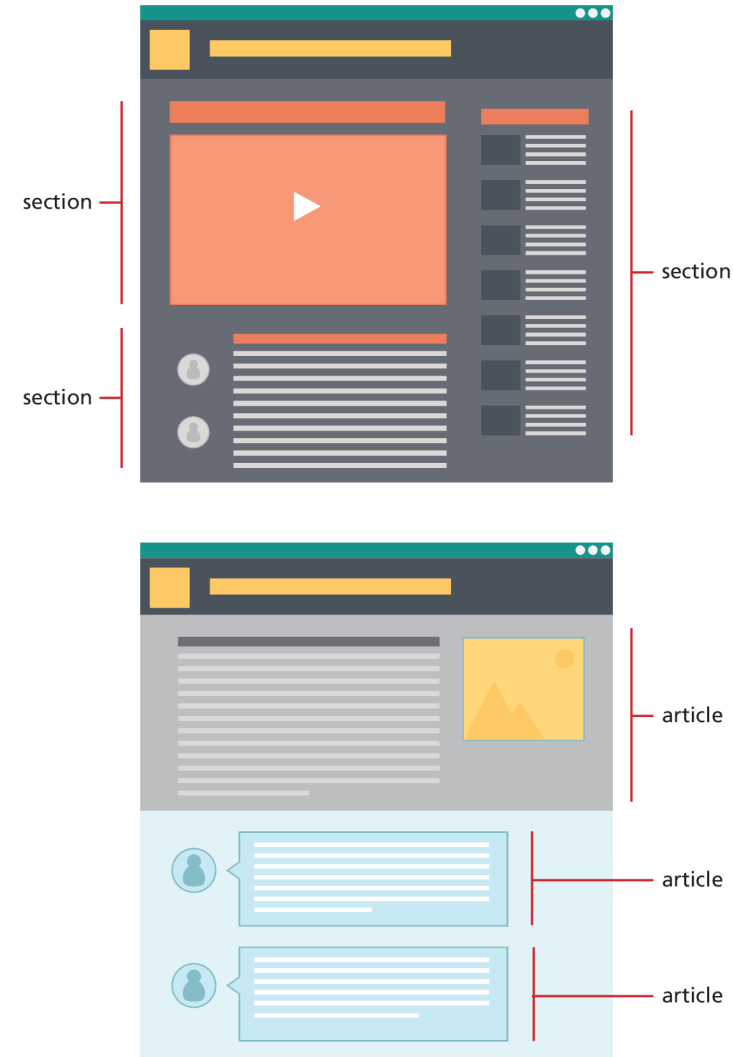


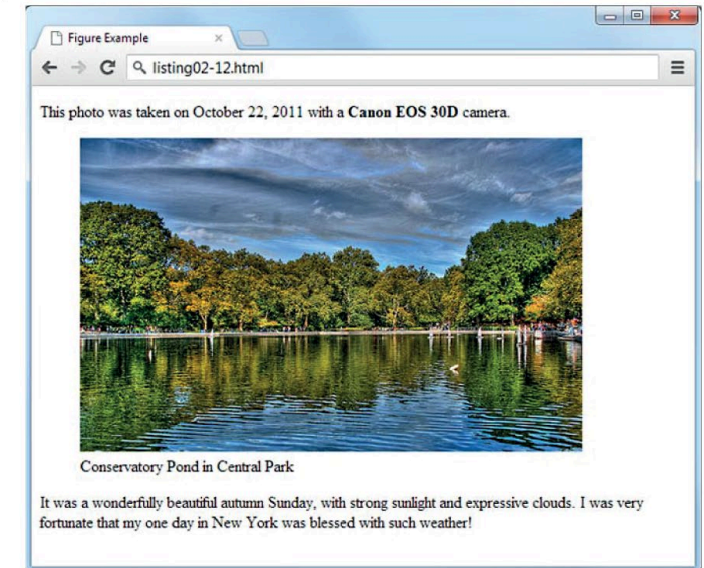
Figure and figcaption

The `<figure>` element can be used not just for images but for any type of *essential* content that could be moved to a different location in the page or document, and the rest of the document would still make sense.

Figure could be moved to a different location in document ...

But it has to exist in the document (i.e., the figure isn't optional).

```
<p>This photo was taken on October 22, 2011 with a Canon EOS 30D camera.</p>
<figure>
  <br/>
  <figcaption>Conservatory Pond in Central Park</figcaption>
</figure>
<p>
  It was a wonderfully beautiful autumn Sunday, with strong sunlight and
  expressive clouds. I was very fortunate that my one day in New York was
  blessed with such weather!
</p>
```



Details and Summary

Clicking on the summary label reveals the rest of the content with the `<details>` container



```

<body>
  <h2>The Milkmaid</h2>
  <details>
    <summary>Image</summary>
    <br>
    <p>Museum: Rijksmuseum, Amsterdam</p>
  </details>
  <details>
    <summary>Artist</summary>
    <p><strong>Jan Vermeer</strong> was a Dutch ...</p>
  </details>
  <details>
    <summary>Information</summary>
    <p>
      Date: 1657<br>
      Medium: Oil on Canvas
    </p>
  </details>
</body>

```

- The `<details>` and `<summary>` elements provide a way of semantically relating a summary and a details.
- For browsers that support these elements, *accordion functionality* is included as well (thus no JavaScript programming is required).

Additional Semantic Elements

- The `<blockquote>` element is a way to indicate a quotation from another source.
- The `<address>` element indicates that the enclosed HTML contains contact information for a person or organization.
- Additional list in Table 3.2

Key Terms

- absolute referencing
- accessibility
- ancestors
- body
- Cascading Style Sheets
- (CSS)
- character entity
- description lists
- descendants
- directory
- document outline
- Document Object Model
- empty element
- folder
- Head
- HTML attribute
- HTML element
- HTML validators
- inline HTML elements
- maintainability
- markup
- markup language
- ordered lists
- pathname
- performance
- polyfill
- quirks mode
- Recommendations
- relative referencing
- root element
- root reference
- schemas
- search engine optimization
- semantic HTML
- specifications
- standards mode
- tags
- unordered lists
- UTF-8
- WHATWG
- World Wide Web Consortium
- W3C
- XHTML 1.0
- XML

Copyright



This work is protected by United States copyright laws and is provided solely for the use of instructors in teaching their courses and assessing student learning. Dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.